INTRODUCCIÓN A LA PROGRAMACIÓN · 1.º GII · PRÁCTICAS

Guía de prácticas g++

Las seis prácticas explicadas: lo que hay que saber de C++ en cada una (con un bloque «**Explicado despacio**» por práctica) y **44 ejercicios resueltos**, cada uno con una **explicación paso a paso** de las fórmulas y del procedimiento (los de clase y los de trabajo personal) con el análisis, la notación SP cuando ayuda, el programa en C++ y una ejecución real. Todo el código está compilado con g++ y probado.

PRÁCTICA 1

Entorno de programación

Geany y g++: editar, compilar y ejecutar un programa C++, y la estructura de un programa.

1.1 Editar, compilar y ejecutar con Geany

En el laboratorio se arranca **Ubuntu** desde el menú de GRUB (usuario y contraseña `alumno`) y se abre Geany desde «Programación → Geany». En casa: Geany (Linux, Windows) o Dev-C++ (Windows), que se usan casi igual. El compilador es **GNU g++**.

Paso	Cómo	Detalles
1. Nuevo	Archivo → Nuevo	Se abre una pestaña «sin título».
2. Guardar	Archivo → Guardar	Nombre acabado en **.cpp**, significativo (`Saludos.cpp`), **sin espacios ni tildes** en el nombre ni en las carpetas. Al guardarlo, Geany reconoce C++ y colorea la sintaxis.
3. Compilar	**F9** (Construir → Construir)	La pestaña «Compilador» dice si ha ido bien; si hay errores, al pulsar en uno se marca la línea (la barra inferior indica línea y columna).
4. Ejecutar	**F5** (Construir → Ejecutar)	Se abre una terminal con la salida; las líneas finales («Pulse Intro...») las añade Geany, no tu programa.

Las cuatro zonas de Geany: barra de menús y botones (arriba), símbolos del archivo (izquierda), edición (centro-derecha) y mensajes / compilador (abajo).

1.2 Estructura de un programa C++

Igual que un algoritmo: **léxico** (bibliotecas con `#include`, tipos, constantes, variables y subprogramas globales) y **algoritmo principal**, que en C++ es la función `main`. `main` devuelve un `int` al sistema operativo: **0 = terminación normal** (si no hay `return`, se devuelve 0). En C++ no se pueden definir funciones dentro de otras.

- **Bloque:** instrucciones entre `{ }`, cada una acabada en `;`.
- **Identificadores:** letras y dígitos empezando por letra (el `_` cuenta como letra, pero no empieces por él). C++ **distingue mayúsculas:** `notaFinal` ≠ `notafinal`.
- **Palabras reservadas** (no se pueden usar como nombres: `using`, `int`…), **literales**, **operadores** y **delimitadores**.
- **Comentarios:** `// hasta el final de la línea` y `/* varias líneas */`. Los blancos y saltos de línea sólo separan.
- `using namespace std;`: cin, cout y el resto de la biblioteca estándar están en el espacio de nombres `std`.

EXPLICADO DESPACIO

Qué pasa cuando pulsas F9

1. **Preprocesador:** sustituye cada `#include` por el contenido de esa biblioteca.
2. **Compilador** (g++): comprueba que el texto es C++ válido y lo traduce a código máquina. Si encuentra un error, para y te dice la línea.
3. **Enlazador:** une tu código con el de las bibliotecas (el de `cout`, por ejemplo) y genera el **ejecutable**.

F5 ejecuta ese ejecutable. Por eso, si cambias el código y no vuelves a compilar, ejecutas la versión vieja.

Cómo leer un mensaje de error

g++ escribe `fichero.cpp:7:5: error: …`: línea 7, columna 5. El error real suele estar en esa línea **o en la anterior** (un `;` que falta al final de la línea 6 se detecta al llegar a la 7). Corrige siempre **el primer error** y vuelve a compilar: muchos de los siguientes son consecuencia de ese.

Notación SP	C++
Léxico	declaraciones (`int x;`) antes de usarlas
Algoritmo … Fin	`int main () { … return 0; }`
←	`=`
Leer (x)	`cin >> x;`
Escribir (x)	`cout << x;`

1.3 Ejercicios

Ejercicio 1 · El primer programa (Saludos.cpp)

Escribe, guarda, compila y ejecuta el programa de la primera página del boletín.

EXPLICACIÓN PASO A PASO

Qué hace cada línea

1. `#include <iostream>` trae la biblioteca de entrada/salida (`cin` , `cout` , `getline` , `string`). Sin ella el compilador no conoce esos nombres.
2. `using namespace std;` evita escribir `std::cout` : todo lo de la biblioteca estándar vive en el espacio de nombres `std` .
3. `int main ()` es el «Algoritmo» de la notación: lo primero que ejecuta el programa. Devuelve un entero al sistema operativo.
4. `string nombre;` es el «Léxico»: declara una variable de tipo cadena.
5. `cout << "¿Cómo te llamas? ";` escribe el mensaje que dice al usuario qué tiene que teclear.
6. `getline (cin, nombre);` lee **toda la línea** hasta el Intro y la guarda en `nombre` .
7. El segundo `cout` encadena tres cosas con `<<` : texto, la variable y más texto; `endl` salta de línea.
8. `return 0;` le dice al sistema operativo «he terminado bien».

El ciclo de trabajo

Escribir → **guardar** como `.cpp` (sin espacios ni tildes en el nombre) → **F9** compilar (g++ traduce el texto a un ejecutable) → **F5** ejecutar. Si cambias el código, hay que volver a compilar: si no, ejecutas la versión anterior.

ERRORES TÍPICOS

Olvidar un `;` al final de una instrucción, escribir `Cout` en vez de `cout` (C++ distingue mayúsculas) o usar comillas tipográficas “ ” copiadas de un PDF en vez de las rectas `"` .

C++ · SALUDOS.CPP

```cpp
#include <iostream>
using namespace std;

int main () {
    string nombre;

    cout << "¿Cómo te llamas? ";
    getline (cin, nombre);
    cout << "Hola, " << nombre << ", vamos a trabajar" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
¿Cómo te llamas? Ana García
Hola, Ana García, vamos a trabajar
```

`getline (cin, nombre)` lee la **línea completa** (nombre y apellidos). Con `cin >> nombre` sólo se leería la primera palabra.

`Ejercicio 2` · **Suma, resta y división de dos enteros**

Tras escribir y ampliar el «segundo programa» (que lee dos números y escribe su suma), modifícalo para que lea dos enteros y muestre la **suma**, la **resta** y la **división**.

EXPLICACIÓN PASO A PASO

La idea

Leer dos enteros y escribir cinco resultados. Lo interesante es que el operador `/` **cambia de significado según el tipo** de los operandos.

Las operaciones, con 17 y 5

- **Suma:** 17 + 5 = 22. **Resta:** 17 − 5 = 12.
- **División entera** (`17 / 5` con dos `int`): cuántas veces cabe el 5 en el 17 sin decimales → **3** (porque 5·3 = 15 ≤ 17 y 5·4 = 20 > 17). Es el **DIV** de la notación.
- **Resto** (`17 % 5`): lo que sobra → 17 − 15 = **2**. Es el **MOD**. Siempre se cumple: dividendo = divisor · cociente + resto (17 = 5·3 + 2).
- **División real:** si uno de los dos es real, `/` hace la división con decimales. `float(numero1)` convierte el 17 en 17.0 y entonces 17.0 / 5 = **3.4**.

Por qué `float(numero1) / numero2` y no `float(numero1 / numero2)`

En la segunda forma primero se hace la división **entera** (da 3) y luego se convierte el 3 en 3.0: el decimal ya se ha perdido. La conversión tiene que ocurrir **antes** de dividir.

C++ · OPERACIONES.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int numero1, numero2;

    cout << "Escribe un número: ";
    cin >> numero1;
    cout << "Escribe otro número: ";
    cin >> numero2;
    cout << "La suma es: " << numero1 + numero2 << endl;
    cout << "La resta es: " << numero1 - numero2 << endl;
    cout << "La división entera es: " << numero1 / numero2 << endl;      // 17 / 5 = 3
    cout << "El resto es: " << numero1 % numero2 << endl;                // 17 % 5 = 2
    cout << "La división real es: " << float(numero1) / numero2 << endl; // 3.4
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Escribe un número: 17
Escribe otro número: 5
La suma es: 22
La resta es: 12
La división entera es: 3
El resto es: 2
La división real es: 3.4
```

OJO CON LA DIVISIÓN

Con dos `int`, `/` es la **división entera** (el DIV de la notación) y `%` el resto (MOD). Para la división real hay que convertir uno de los operandos: `float(numero1) / numero2`. Si el segundo número es 0, la división entera hace que el programa termine con error: más adelante (práctica 3) se puede comprobar con un `if`.

Ejercicio 3 · **Algoritmo 2.1: nota final**

Codifica el algoritmo 2.1 del libro: la nota final es el 70 % de teoría y el 30 % de prácticas.

EXPLICACIÓN PASO A PASO

De dónde sale la fórmula

Una media **ponderada** da a cada nota un peso: el 70 % es 70/100 = **0.7** y el 30 % es **0.3**. Los pesos suman 1 (0.7 + 0.3 = 1), así que si las dos notas fueran iguales la nota final sería esa misma nota.

```
notaFinal = notaTeoria · 0.7 + notaPractica · 0.3
```

Procedimiento

1. **Datos:** dos reales (la nota puede tener decimales) → `float` .
2. **Resultado:** otro real.
3. Leer las dos notas con un solo `cin >> notaTeoria >> notaPractica;` : el usuario puede teclearlas separadas por un blanco o en dos líneas.
4. Calcular y escribir.

Comprobación: 6.5 y 8 → 6.5·0.7 = 4.55; 8·0.3 = 2.4; total **6.95**. Otra prueba rápida: 10 y 0 → 7, porque la teoría pesa el 70 %.

> EN C++ EL DECIMAL ES UN PUNTO
> Se escribe `0.7` , no `0,7` . Con coma, el compilador lo interpreta como dos cosas distintas y la fórmula sale mal sin avisar.

C++ · NOTA.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    float notaTeoria;
    float notaPractica;
    float notaFinal;

    cout << "Introduce la nota de teoría y la de prácticas: ";
    cin >> notaTeoria >> notaPractica;
    notaFinal = notaTeoria * 0.7 + notaPractica * 0.3;
    cout << notaFinal << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Introduce la nota de teoría y la de prácticas: 6.5 8
6.95
```

6.5 · 0.7 + 8 · 0.3 = 4.55 + 2.4 = **6.95**. Fíjate en la correspondencia con la notación: `float` = Real, `cin >>` = Leer, `=` = ←, `cout <<` = Escribir.

PRÁCTICA 2

Léxico y estados

Variables y constantes de un problema, tipos, secuenciación, estados intermedios, asignación y entrada/salida.

2.1 Tipos, operadores y conversiones

Notación	C++	Rango / tamaño típico
Entero	`short`, `int`, `long`, `long long`	`int`: −2 147 483 648 … 2 147 483 647 (4 bytes); `short`: −32 768 … 32 767 (2)
Real	`float`, `double`	Aproximados: cada valor representa un intervalo de la recta real
Booleano	`bool` (`true`, `false`)	—
Carácter	`char`	−128 … 127 (1 byte); `int ('B')` = 66, `char (66)` = 'B'

Notación	×	DIV	MOD	/	≠	=	Y	O	NO
C++	`*`	`/`	`%`	`/`	`!=`	`==`	`&&`	`||`	`!`

REALES: NO COMPARES CON ==

Los cálculos con reales son aproximados. Con `float`, `1.0 - 0.2 - 0.8` da 0 pero `1.0 - 0.8 - 0.2` da $-1.49 \cdot 10^{-8}$. En vez de `a == b`, comprueba `fabs (a - b) < EPS` con una constante pequeña.

Conversiones implícitas (evítalas): con `int i; bool text;`, `i = 3.6;` deja 3 y `text = 'A';` deja `true` (se escribe 1). Lo único admitido en la notación es asignar una expresión entera a una variable real.

Constantes: `const <tipo> <nombre> = <valor>;`. Ojo: el anexo pone `const int PI = 3.14159;`, que vale 3 por ser `int`; debe ser `const float PI = 3.14159;`.

EXPLICADO DESPACIO

Por qué los tipos tienen rango

Un `int` ocupa 4 bytes = 32 bits, y con 32 bits en complemento a 2 sólo caben los valores de -2^{31} a $2^{31} - 1$ (−2 147 483 648 … 2 147 483 647). Si una cuenta se sale, el resultado «da la vuelta» y sale un número absurdo **sin ningún aviso**: es el desbordamiento. Por eso para factoriales o sucesiones que crecen rápido se usa `long long` (8 bytes).

Cómo decide C++ qué división hacer

- `int / int` → división **entera**: 7 / 2 = 3.
- Si uno de los dos es real → división **real**: 7.0 / 2 = 3.5; `float(7) / 2` = 3.5.
- `%` sólo existe entre enteros: 7 % 2 = 1. Con negativos, el resto lleva el signo del dividendo: −7 % 2 = −1.

Prioridad de los operadores (de más a menos)

`!` → `* / %` → `+ -` → `< <= > >=` → `== !=` → `&&` → `||`. Ante la duda, pon paréntesis: `(a + b) / 2` no es lo mismo que `a + b / 2`.

Por qué no se comparan reales con ==

Muchos decimales no tienen representación exacta en binario (0.1 es periódico en base 2, igual que 1/3 en base 10). Cada operación redondea un poco y los errores se acumulan, así que dos cálculos «iguales» pueden diferir en la última cifra. Por eso se pregunta «¿están muy cerca?»: `fabs (a - b) < EPS`, con EPS pequeño (0.0001, por ejemplo).

2.2 Entrada y salida: el buffer del teclado

`cin >>` toma los valores del **buffer** del teclado (si está vacío, espera a que se pulse Intro). Se salta los blancos iniciales y convierte el texto en número automáticamente.

Buffer	Lecturas	Resultado
`1234␣␣␣␣⏎`	`int v; cin >> v;`	v = 1234 (quedan 4 blancos y el ⏎ en el buffer)
`1234⏎`	`float v; cin >> v;`	v = 1234.0
`1234␣␣␣␣⏎`	`char c; int i; cin >> c; cin >> i;`	c = '1', i = 234
`La Luna del Valle.⏎`	tres `cin >> c` con char	'L', 'a', 'L' (el blanco se salta); queda «una del Valle.⏎»

- Para leer **todos** los caracteres, incluidos blancos y saltos de línea: `cin.get (c)` (prácticas 5 y 6).
- Para leer una **línea** en un `string`: `getline (cin, cadena)`; `cin >> cadena` lee sólo una palabra.
- En programas interactivos, cada lectura debe ir precedida de un mensaje que diga qué hay que teclear.
- La asignación es `=`; sólo puede haber una variable a la izquierda, y deben ser del mismo tipo (salvo entero → real).

EXPLICADO DESPACIO

Cómo funciona el buffer, paso a paso

1. Lo que tecleas no llega al programa letra a letra: se acumula en el **buffer** hasta que pulsas Intro.
2. Cada `cin >> v` se salta los blancos y saltos de línea que haya al principio y toma **los caracteres que encajan con el tipo de v**: dígitos para un `int`, dígitos y un punto para un `float`, un solo carácter para un `char`.
3. Lo que no ha tomado **se queda en el buffer** para la siguiente lectura.
4. Si el buffer está vacío, el programa se para a esperar otra línea.

Por eso con «1234␣␣␣␣⏎» y `char c; int i; cin >> c >> i;` sale c = '1' (un char es un solo carácter) e i = 234 (el resto de los dígitos).

> **LA TRAMPA DE MEZCLAR `>>` Y `GETLINE`**
> Tras `cin >> edad;` el ⏎ se queda en el buffer. Un `getline (cin, nombre);` justo después lee esa línea vacía y parece que «se salta» la pregunta. Solución: `cin.ignore();` antes del `getline`, o leer todo con `getline`.

2.3 Ejercicios de clase

Ejercicio 1 · **Intercambio circular de tres variables**

Dadas A, B y C: B toma el valor de A, C el de B y A el de C. Haz una traza, piensa qué pasa si se cambia el orden de las asignaciones y codifícalo mostrando los estados intermedios.

EXPLICACIÓN PASO A PASO

El problema de fondo

Una asignación `b ← a` **copia** el valor de a en b y **borra** lo que había en b. Si queremos mover tres valores en círculo, al hacer la primera asignación perdemos un valor que todavía necesitamos. Por eso hace falta una variable **auxiliar** que lo guarde antes.

Cómo se construye el orden correcto

1. Lo que queremos al final: b = $a_{inicial}$, c = $b_{inicial}$, a = $c_{inicial}$.
2. Elige qué variable vas a pisar primero. Si empiezas por `b ← a`, se pierde $b_{inicial}$, que es justo lo que necesita c. Así que guárdalo antes: `aux ← b`.
3. Ahora ya puedes hacer `b ← a`.
4. a ya está copiada en b, así que puedes pisar a con c: `a ← c`.
5. Por último c recibe el b inicial, que está a salvo en aux: `c ← aux`.

La regla general: **antes de pisar una variable, asegúrate de que su valor está guardado en otro sitio o ya no hace falta.** Con n variables en círculo basta una sola auxiliar.

Qué es un «estado»

El estado es la foto de los valores de todas las variables en un momento. La traza (la tabla e1…e5) es la secuencia de fotos tras cada instrucción, y sirve para comprobar el algoritmo sin ordenador. El programa en C++ las imprime para que veas que coinciden con la traza a mano.

NOTACIÓN SP

```
Léxico
  a, b, c : Carácter;   (datos de entrada y de salida)
  aux : Carácter;       (variable auxiliar)
Algoritmo
  Leer (a, b, c);   {e1: los valores introducidos}
  aux ← b;          {e2: aux = b}
  b ← a;            {e3: b = a}
  a ← c;            {e4: a = c inicial, b = a inicial}
  c ← aux;          {e5: desplazamiento circular hecho}
  Escribir (a, b, c)
Fin
```

Traza con a = 'x', b = 'y', c = 'z'

Estado	a	b	c	aux
e1	x	y	z	?
e2	x	y	z	y
e3	x	x	z	y
e4	z	x	z	y
e5	z	x	y	y

¿Y si se cambia el orden? Cada asignación **destruye** el valor anterior de la variable. Por ejemplo, si se hace `b ← a` antes de `aux ← b`, el valor original de b se pierde y al final c recibe el de a. El orden correcto guarda primero (en aux) el valor que se va a pisar.

C++ · INTERCAMBIO.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    char a, b, c;   // datos de entrada y de salida
    char aux;       // variable auxiliar

    cout << "Introduce tres caracteres: ";
    cin >> a >> b >> c;
    cout << "e1: a=" << a << " b=" << b << " c=" << c << endl;
    aux = b;
    cout << "e2: a=" << a << " b=" << b << " c=" << c << " aux=" << aux << endl;
    b = a;
    cout << "e3: a=" << a << " b=" << b << " c=" << c << " aux=" << aux << endl;
    a = c;
    cout << "e4: a=" << a << " b=" << b << " c=" << c << " aux=" << aux << endl;
    c = aux;
    cout << "e5: a=" << a << " b=" << b << " c=" << c << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Introduce tres caracteres: x y z
e1: a=x b=y c=z
e2: a=x b=y c=z aux=y
e3: a=x b=x c=z aux=y
e4: a=z b=x c=z aux=y
e5: a=z b=x c=y
```

Ejercicio 2 · **Desglose de dinero**

Desglosa una cantidad en el mínimo número de billetes de 500, 100, 50, 20, 10 y 5 euros; el resto, en monedas.

EXPLICACIÓN PASO A PASO

La idea: el método voraz

Para usar el **mínimo número** de billetes se empieza por el más grande: se ponen todos los billetes de 500 que quepan y, con lo que sobra, se pasa al siguiente. «Cuántos caben» es la **división entera** y «lo que sobra» es el **resto**.

```
billetes = cantidad DIV valor · resto = cantidad MOD valor
```

Traza con 1789 €

Billete	Cuenta	Cuántos	Resto
500	1789 DIV 500 · 1789 MOD 500	3	289
100	289 DIV 100 · 289 MOD 100	2	89
50	89 DIV 50	1	39
20	39 DIV 20	1	19
10	19 DIV 10	1	9
5	9 DIV 5	1	4 (monedas)

Comprobación: 3·500 + 2·100 + 50 + 20 + 10 + 5 + 4 = 1500 + 200 + 89 = 1789 ✓. Es el mismo patrón que pasar segundos a días, horas, minutos y segundos, con 86400, 3600 y 60 en lugar de los billetes.

DOS ÓRDENES QUE IMPORTAN

Dentro de cada billete, primero el **cociente** y después el **resto** (si actualizas resto primero, el cociente se calcula con un número ya reducido). Y los billetes, de **mayor a menor**.

Misma estrategia que «segundos → días, horas, minutos y segundos» del tema 1: cociente (DIV) por el billete más grande y seguir con el resto (MOD).

NOTACIÓN SP

```
Leer (cantidad);
quinientos ← cantidad DIV 500;   resto ← cantidad MOD 500;
cien ← resto DIV 100;            resto ← resto MOD 100;
cincuenta ← resto DIV 50;        resto ← resto MOD 50;
veinte ← resto DIV 20;           resto ← resto MOD 20;
diez ← resto DIV 10;             resto ← resto MOD 10;
cinco ← resto DIV 5;             resto ← resto MOD 5;
Escribir (quinientos, cien, cincuenta, veinte, diez, cinco, resto)
```

C++ · DESGLOSE.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int cantidad, resto;
    int quinientos, cien, cincuenta, veinte, diez, cinco;

    cout << "Cantidad en euros: ";
    cin >> cantidad;
    quinientos = cantidad / 500;   resto = cantidad % 500;
    cout << "  quinientos=" << quinientos << " resto=" << resto << endl;
    cien = resto / 100;            resto = resto % 100;
    cout << "  cien=" << cien << " resto=" << resto << endl;
    cincuenta = resto / 50;        resto = resto % 50;
    cout << "  cincuenta=" << cincuenta << " resto=" << resto << endl;
    veinte = resto / 20;           resto = resto % 20;
    cout << "  veinte=" << veinte << " resto=" << resto << endl;
    diez = resto / 10;             resto = resto % 10;
    cout << "  diez=" << diez << " resto=" << resto << endl;
    cinco = resto / 5;             resto = resto % 5;
    cout << "  cinco=" << cinco << " resto=" << resto << endl;

    cout << quinientos << " de quinientos euros" << endl;
    cout << cien << " de cien euros" << endl;
    cout << cincuenta << " de cincuenta euros" << endl;
    cout << veinte << " de veinte euros" << endl;
    cout << diez << " de diez euros" << endl;
    cout << cinco << " de cinco euros" << endl;
    cout << resto << " en monedas" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Cantidad en euros: 1789
  quinientos=3 resto=289
  cien=2 resto=89
  cincuenta=1 resto=39
  veinte=1 resto=19
  diez=1 resto=9
  cinco=1 resto=4
3 de quinientos euros
2 de cien euros
1 de cincuenta euros
1 de veinte euros
1 de diez euros
1 de cinco euros
4 en monedas
```

¿Y si se cambia el orden? Si se calcula `resto` antes que el cociente del mismo billete, el cociente se calcula con un resto ya reducido (sale 0). Y si se empieza por los billetes pequeños deja de ser el número mínimo de billetes (1789 serían 357 de 5).

Ejercicio 3 · **Binario a decimal**

Dado un binario de cuatro cifras (cuatro enteros 0 o 1, por ejemplo 1, 1, 0, 1), escribe su valor en base 10. ¿Qué variables hacen falta?

EXPLICACIÓN PASO A PASO

De dónde salen los pesos

En un número de base 2 cada cifra vale **cifra** × $2^{\text{posición}}$, contando posiciones desde 0 por la derecha. Con cuatro cifras b3 b2 b1 b0 los pesos son $2^3 = 8$, $2^2 = 4$, $2^1 = 2$ y $2^0 = 1$.

```
decimal = b3·8 + b2·4 + b1·2 + b0·1
```

Procedimiento

1. **Variables**: cuatro enteros para las cifras, leídas de la más significativa (izquierda) a la menos significativa, y uno para el resultado.
2. Leer las cuatro cifras en ese orden.
3. Aplicar la fórmula y escribir el resultado.

Traza con 1, 1, 0, 1: 1·8 + 1·4 + 0·2 + 1·1 = 8 + 4 + 0 + 1 = **13**. Con 1111 sale el máximo, 15; con 0000, 0: con 4 bits hay $2^4 = 16$ valores, del 0 al 15.

> Si lees las cifras en el orden contrario, el programa da otro número (1101 leído al revés es 1011 = 11). Decide el orden y explícalo en el mensaje al usuario.

Variables: cuatro enteros para las cifras (de la más significativa a la menos) y uno para el resultado. Cada cifra se multiplica por su peso: 8, 4, 2, 1.

NOTACIÓN SP

```
Léxico
  b3, b2, b1, b0 : Entero [0, 1];
  decimal : Entero [0, 15];
Algoritmo
  Leer (b3, b2, b1, b0);
  decimal ← b3 * 8 + b2 * 4 + b1 * 2 + b0;
  Escribir (decimal)
Fin
```

C++ · BINARIO.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int b3, b2, b1, b0;   // cifras binarias, de la más significativa a la menos
    int decimal;

    cout << "Introduce las cuatro cifras binarias separadas por espacios: ";
    cin >> b3 >> b2 >> b1 >> b0;
    decimal = b3 * 8 + b2 * 4 + b1 * 2 + b0;
    cout << "En base 10 vale " << decimal << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Introduce las cuatro cifras binarias separadas por espacios: 1 1 0 1
En base 10 vale 13
```

$1101_2 = 8 + 4 + 0 + 1 = 13$.

2.4 Trabajo personal

Problema 1 · **JetLag: coste de un vuelo**

Por cada vuelo se anotan los kilómetros y los litros gastados; el precio del litro es propio de cada vuelo y hay un gasto fijo de mantenimiento por kilómetro. Calcula el coste total, los kilómetros por litro y el coste por kilómetro.

EXPLICACIÓN PASO A PASO

Separar datos y resultados

Qué	Variables	Unidades
Datos	km, litros, precioLitro, gastoKm	km, l, €/l, €/km
Resultados	coste, kmPorLitro, costeKm	€, km/l, €/km

Cómo se deduce cada fórmula (mirando las unidades)

- **Coste:** hay dos gastos. El combustible: litros × €/litro = **€**. El mantenimiento: km × €/km = **€**. Se suman: coste = litros · precioLitro + km · gastoKm.
- **Kilómetros por litro:** «por» significa dividir: km / litros.
- **Coste por kilómetro:** € / km = coste / km. Se calcula **después** de coste, porque lo usa.

Truco: si las unidades de la fórmula no dan las del resultado, la fórmula está mal.

Comprobación con 1200 km, 3000 l, 0.85 €/l, 1.5 €/km

Combustible: 3000 · 0.85 = 2550 €. Mantenimiento: 1200 · 1.5 = 1800 €. Coste = **4350 €**. km/l = 1200 / 3000 = **0.4**. €/km = 4350 / 1200 = **3.625**.

- **Entrada:** km, litros, precioLitro, gastoKm (todos reales).
- **Salida:** coste = litros · precioLitro + km · gastoKm; kmPorLitro = km / litros; costeKm = coste / km.

NOTACIÓN SP

```
Léxico
  km, litros, precioLitro, gastoKm : Real;    // datos
  coste, kmPorLitro, costeKm : Real;          // resultados
Algoritmo
  Leer (km, litros, precioLitro, gastoKm);
  coste ← litros * precioLitro + km * gastoKm;
  kmPorLitro ← km / litros;
  costeKm ← coste / km;
  Escribir (coste, kmPorLitro, costeKm)
Fin
```

Traza a mano con 1200 km, 3000 l, 0.85 €/l y 1.5 €/km: coste = 2550 + 1800 = 4350 €, 0.4 km/l, 3.625 €/km.

C++ · JETLAG.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    float km, litros, precioLitro, gastoKm;   // datos de un vuelo
    float coste, kmPorLitro, costeKm;         // resultados

    cout << "Kilómetros recorridos: ";              cin >> km;
    cout << "Litros de combustible: ";              cin >> litros;
    cout << "Precio del litro en este vuelo: ";     cin >> precioLitro;
    cout << "Gasto fijo de mantenimiento por km: "; cin >> gastoKm;

    coste = litros * precioLitro + km * gastoKm;
    kmPorLitro = km / litros;
    costeKm = coste / km;

    cout << "Coste total del viaje: " << coste << " euros" << endl;
    cout << "Kilómetros por litro: " << kmPorLitro << endl;
    cout << "Coste por kilómetro: " << costeKm << " euros" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Kilómetros recorridos: 1200
Litros de combustible: 3000
Precio del litro en este vuelo: 0.85
Gasto fijo de mantenimiento por km: 1.5
Coste total del viaje: 4350 euros
Kilómetros por litro: 0.4
Coste por kilómetro: 3.625 euros
```

Problema 2 · **Nota final ponderada (50 %, 30 %, 20 %)**

Dadas las notas del examen, del trabajo y del test, muestra la nota final.

EXPLICACIÓN PASO A PASO

Igual que el algoritmo 2.1, con tres pesos

Los porcentajes se pasan a tanto por uno: 50 % = 0.5, 30 % = 0.3, 20 % = 0.2. Suman 1, así que el resultado sigue estando entre 0 y 10.

```
final = examen · 0.5 + trabajo · 0.3 + test · 0.2
```

1. Leer las tres notas como reales.
2. Multiplicar cada una por su peso y sumar.
3. Escribir el resultado.

Comprobación con 6, 8 y 5: 6·0.5 = 3; 8·0.3 = 2.4; 5·0.2 = 1; total **6.4**. Fíjate en que el trabajo (8) sube la nota más que el test (5) la baja, porque pesa más.

Entrada: tres reales. Salida: `examen · 0.5 + trabajo · 0.3 + test · 0.2`. Con 6, 8 y 5: 3 + 2.4 + 1 = 6.4.

C++ · PONDERADA.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    float examen, trabajo, test, notaFinal;

    cout << "Nota del examen, del trabajo y del test: ";
    cin >> examen >> trabajo >> test;
    notaFinal = examen * 0.5 + trabajo * 0.3 + test * 0.2;
    cout << "Nota final: " << notaFinal << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Nota del examen, del trabajo y del test: 6 8 5
Nota final: 6.4
```

Problema 3 · **Contenedores para cerdos ibéricos**

Hay contenedores para 50, 25, 5 y 1 cerdos. Dado el número de animales, calcula cuántos contenedores de cada tipo hacen falta usando el menor número posible y, si sobra espacio, que sea el menor posible.

EXPLICACIÓN PASO A PASO

Por qué es «el desglose de dinero»

Los contenedores de 50, 25, 5 y 1 hacen el papel de los billetes. Se llena primero el más grande todo lo posible (DIV) y se sigue con los animales que quedan (MOD).

Traza con 138 cerdos

Contenedor	DIV	MOD (quedan)
50	138 DIV 50 = 2	38
25	38 DIV 25 = 1	13
5	13 DIV 5 = 2	3
1	3 DIV 1 = 3	0

138 = 2·50 + 1·25 + 2·5 + 3·1 ✓: 8 contenedores.

Las dos condiciones del enunciado

- **Que sobre el menor espacio posible**: como existe el contenedor de 1, el último paso siempre deja 0: no sobra nada.
- **El menor número de contenedores**: empezar por el grande funciona aquí porque cada tamaño «encaja» en los siguientes (50 = 2·25, 25 = 5·5, 5 = 5·1). Con tamaños raros (por ejemplo 6, 4 y 1 para 8 animales) el método voraz fallaría: daría 6 + 1 + 1 (3 contenedores) en vez de 4 + 4 (2).

Es el desglose de dinero con otros «billetes». Como existe el contenedor de 1, el reparto es exacto y **no sobra espacio**; el método voraz (empezar por el más grande) da además el mínimo número de contenedores, porque cada tamaño es múltiplo del siguiente.

C++ · CERDOS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int cerdos, resto;
    int c50, c25, c5, c1;   // contenedores de cada tamaño

    cout << "Número de cerdos: ";
    cin >> cerdos;
    c50 = cerdos / 50;  resto = cerdos % 50;
    c25 = resto / 25;   resto = resto % 25;
    c5 = resto / 5;     resto = resto % 5;
    c1 = resto;
    cout << c50 << " de 50, " << c25 << " de 25, " << c5 << " de 5 y " << c1 << " de 1" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Número de cerdos: 138
2 de 50, 1 de 25, 2 de 5 y 3 de 1
```

138 = 2·50 + 1·25 + 2·5 + 3·1.

PRÁCTICA 3

Análisis de casos

if, if–else y switch en C++, expresiones booleanas y problemas con análisis de casos.

3.1 De SI y SEGÚN a if y switch

NOTACIÓN

```
SI cond ENTONCES S FIN_SI
SI cond ENTONCES S SI_NO R FIN_SI
SEGÚN c1, …, cn
  e1 : S1;
  e2 : S2;
  …
  EN_OTRO_CASO : Sn+1
FIN_SEGÚN
```

C++

```
if (cond) { S }
if (cond) { S } else { R }
if (e1) { S1; }
else if (e2) { S2; }
…
else { Sn+1; }
```

El `switch` se parece al SEGÚN pero es más limitado: selecciona según el valor de **una única expresión** de **tipo ordinal** (entero, char, bool…) comparada con valores **estáticos** (literales o constantes conocidas al compilar). Cuando entra en un `case` ejecuta todo lo que encuentra hasta un `break`; así se asocian varios valores a un mismo tratamiento.

SWITCH

```
switch (PermisoConducir) {
    case 'A':
        cout << "Motocicletas y triciclos de motor"; break;
    case 'B':
        cout << "Automóviles masa máxima autorizada ⩽ 3500 kg"; break;
    case 'C':
        cout << "Automóviles masa máxima autorizada > 3500 kg"; break;
    default:
        cout << "No contemplado"; break;
}
```

ERRATAS DEL BOLETÍN

- Los ejemplos usan comillas tipográficas (‘A’, “No contemplado") que el compilador no acepta: escribe siempre `'A'` y `"…"` rectas.
- En el ejemplo de `if-else` la rama de las mayúsculas aparece dos veces; la segunda nunca se ejecuta (casos solapados).

EXPLICADO DESPACIO

Cómo se diseña un análisis de casos

1. **Encuentra los casos**: pregúntate en qué situaciones el resultado se calcula de forma distinta (la fecha que cambia de mes, el préstamo con recargo…).
2. **Escribe la condición de cada caso** como una expresión booleana.
3. **Comprueba que cubren todo** (cualquier dato válido cae en algún caso) y que **no se solapan** (ninguno cae en dos).
4. **Elige la estructura**: `if` si hay un caso y «nada»; `if … else` si hay dos; una cadena `if … else if … else` si hay varios; `switch` si se compara **un** valor entero o carácter con constantes.

Por qué una cadena `else if` simplifica las condiciones

Cada `else if` sólo se evalúa si todas las condiciones anteriores han sido falsas, así que puede **dar por sabido** lo que ya se descartó. En las calificaciones, al llegar a `else if (nota < 7)` ya se sabe que nota ≥ 5: no hace falta escribir `nota ⩾ 5 && nota < 7`.

Las reglas del `switch`

- La expresión tiene que ser entera o carácter (no `float` ni `string`).
- Cada `case` lleva una constante. Varios `case` seguidos comparten código.
- `break` termina el switch: sin él, se sigue ejecutando el código del caso de abajo.
- `default` es el EN_OTRO_CASO.

3.2 Ejercicios de clase

Problema 1 · Algoritmos 2.8 (calificación) y 2.10 (máximo)

Transcribe a C++ los dos algoritmos y comprueba su comportamiento con varias ejecuciones.

EXPLICACIÓN PASO A PASO

Por qué `switch` en el 2.8

`switch` sirve cuando se compara **una sola expresión de tipo ordinal** (entero, carácter) con **valores constantes**. La nota sobre 20 es un entero y cada calificación corresponde a valores concretos: es el caso ideal.

1. `switch (nota)` evalúa la nota una vez.
2. Salta al `case` con ese valor. Varios `case` seguidos sin nada entre ellos comparten instrucciones: `case 19: case 18:` = «19 o 18».
3. `break` termina el switch. Sin él, la ejecución «caería» al caso siguiente y escribiría también su calificación.
4. `default` recoge todo lo demás (de 0 a 13, y también valores imposibles como 25).

El 2.10: SI–SI_NO

Con dos números hay dos casos: n1 > n2 (el máximo es n1) y el resto (n1 ≤ n2: el máximo es n2). Juntos cubren todas las posibilidades y no se solapan. Si son iguales, cae en el SI_NO y escribe n2, que vale lo mismo.

Cómo probar un análisis de casos: un valor de cada caso y todos los **límites** (20, 18, 14, 13), porque los fallos aparecen justo en los bordes (un `<` que debía ser `⩽`).

El 2.8 compara una única expresión ordinal (la nota entera) con constantes: es el caso ideal para `switch`, agrupando valores con `case` consecutivos. El 2.10 es un SI–SI_NO.

C++ · ALG28.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int nota;

    cout << "Nota (sobre 20): ";
    cin >> nota;
    switch (nota) {
        case 20:
            cout << "M. de Honor" << endl; break;
        case 19:
        case 18:
            cout << "Sobresaliente" << endl; break;
        case 17:
        case 16:
            cout << "Notable" << endl; break;
        case 15:
        case 14:
            cout << "Aprobado" << endl; break;
        default:
            cout << "Suspenso" << endl; break;
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Nota (sobre 20): 17
Notable
```

C++ · ALG210.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int n1, n2;

    cin >> n1 >> n2;
    if (n1 > n2) {
        cout << "El máximo es " << n1 << endl;
    } else {
        cout << "El máximo es " << n2 << endl;
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
4 9
El máximo es 9
```

Pruébalos con los casos frontera: 20, 18, 14, 13 y un valor fuera de rango (25 → «Suspenso» por el default); y en el máximo, con dos números iguales (escribe el segundo, que es el mismo valor).

Problema 2 · **Préstamo rápido**

Un préstamo de 1 a 24 meses: interés total = cantidad · meses · interés mensual. Si el cliente tiene otras deudas se le aplica un recargo del 1 % sobre cantidad + intereses. El total se paga en cuotas mensuales iguales. Calcula la cuota.

EXPLICACIÓN PASO A PASO

De dónde sale cada fórmula

- El interés es **simple**: cada mes se paga el mismo porcentaje sobre la cantidad prestada. Si el interés es del 1.5 % mensual, cada mes se pagan 1000 · 1.5/100 = 15 €; en 10 meses, 150 €. Por eso: **interesTotal = cantidad · meses · interes / 100**. Se divide entre 100 porque el interés se teclea en tanto por ciento.
- Lo que se devuelve en total es lo prestado más los intereses: **total = cantidad + interesTotal**.
- El recargo del 1 % se aplica sobre ese total: total + total · 0.01 (que es lo mismo que total · 1.01).
- Cuotas iguales: **cuota = total / meses**.

Procedimiento

1. Leer cantidad, meses, interés y si tiene otras deudas (S/N) y convertir la respuesta en un booleano: `riesgo = (respuesta == 'S') || (respuesta == 's');`
2. Calcular interesTotal y total.
3. **Sólo si** hay riesgo, aplicar el recargo: es un SI sin SI_NO, porque en el otro caso no se hace nada.
4. Calcular y escribir la cuota.

Comprobación con 1000 €, 10 meses, 1.5 % y deudas: interés 1000·10·1.5/100 = 150 → total 1150 → recargo 11.5 → 1161.5 → cuota 1161.5/10 = **116.15 €**. Sin deudas sería 115 €.

ORDEN DE LAS OPERACIONES

El recargo va **antes** de dividir entre los meses; si se aplica a la cuota da lo mismo aquí (es un porcentaje), pero si el enunciado fuera un recargo fijo en euros ya no daría igual. Sigue el orden del enunciado.

NOTACIÓN SP

```
Léxico
  cantidad, interes : Real;      // interés mensual en %
  meses : Entero [1, 24];
  riesgo : Booleano;
  interesTotal, total, cuota : Real;
Algoritmo
  Leer (cantidad, meses, interes, riesgo);
  interesTotal ← cantidad * meses * interes / 100;
  total ← cantidad + interesTotal;
  SI riesgo ENTONCES total ← total + total * 0.01 FIN_SI;
  cuota ← total / meses;
  Escribir (cuota)
Fin
```

C++ · PRESTAMO.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const float RECARGO = 0.01;     // 1 % para clientes de riesgo
    float cantidad, interes;        // euros y % mensual
    int meses;                      // entre 1 y 24
    char respuesta;
    bool riesgo;
    float interesTotal, total, cuota;

    cout << "Cantidad prestada: ";              cin >> cantidad;
    cout << "Número de meses (1-24): ";         cin >> meses;
    cout << "Interés mensual (%): ";            cin >> interes;
    cout << "¿Tiene otras deudas? (S/N): ";     cin >> respuesta;
    riesgo = (respuesta == 'S') || (respuesta == 's');

    interesTotal = cantidad * meses * interes / 100;
    total = cantidad + interesTotal;
    if (riesgo) {
        total = total + total * RECARGO;
    }
    cuota = total / meses;
    cout << "Cuota mensual: " << cuota << " euros" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Cantidad prestada: 1000
Número de meses (1-24): 10
Interés mensual (%): 1.5
¿Tiene otras deudas? (S/N): S
Cuota mensual: 116.15 euros
```

1000 · 10 · 1.5 / 100 = 150; total 1150; con recargo 1161.5; cuota 116.15 €. El interés se lee en tanto por ciento, por eso se divide entre 100.

Problema 3 · **Fecha del día siguiente**

Lee una fecha (día, mes, año) y escribe la del día siguiente. El año no es bisiesto. Ejemplos: 29 6 2005 → 30 6 2005; 30 4 1990 → 1 5 1990; 31 12 2010 → 1 1 2011.

EXPLICACIÓN PASO A PASO

Cómo se encuentran los casos

Pregúntate: **¿cuándo «salta» la fecha de otra manera?** Normalmente sólo sube el día. Pero el último día del mes el día vuelve a 1 y sube el mes; y el último día del año, además, el mes vuelve a 1 y sube el año. Eso da tres casos.

1. **Primero**, saber cuántos días tiene el mes: 30 para abril, junio, septiembre y noviembre; 28 para febrero (no bisiesto); 31 para el resto. Es un `switch` sobre el mes, con `default` para los de 31.
2. **Caso 1:** dia < diasMes → dia + 1.
3. **Caso 2:** es el último día (dia = diasMes) de un mes que no es diciembre → dia = 1, mes + 1.
4. **Caso 3:** es el 31 de diciembre → dia = 1, mes = 1, año + 1.

Por qué la cadena `if … else if … else` basta

Si no se cumple el caso 1, ya sabemos que dia = diasMes, así que el segundo `if` sólo tiene que mirar el mes; y si tampoco, sólo queda diciembre. Cada pregunta aprovecha lo que ya se sabe de las anteriores.

Comprobación con los tres ejemplos: 29/6 → junio tiene 30, 29 < 30 → 30/6. 30/4 → abril tiene 30, es el último, abril < 12 → 1/5. 31/12/2010 → último de diciembre → 1/1/2011.

Análisis de casos guiado por los datos: (1) no es el último día del mes; (2) es el último día de un mes que no es diciembre; (3) es el 31 de diciembre. Antes se calcula cuántos días tiene el mes (un `switch` sobre el mes).

NOTACIÓN SP

```
Leer (dia, mes, anyo);
SEGÚN mes
  mes = 4, 6, 9, 11 : diasMes ← 30;
  mes = 2           : diasMes ← 28;
  EN_OTRO_CASO      : diasMes ← 31
FIN_SEGÚN;
SEGÚN dia, mes
  dia < diasMes                  : dia ← dia + 1;
  (dia = diasMes) Y (mes < 12)   : dia ← 1; mes ← mes + 1;
  (dia = diasMes) Y (mes = 12)   : dia ← 1; mes ← 1; anyo ← anyo + 1
FIN_SEGÚN;
Escribir (dia, mes, anyo)
```

C++ · FECHA.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int dia, mes, anyo;
    int diasMes;   // días que tiene el mes leído (año no bisiesto)

    cout << "Fecha (día mes año): ";
    cin >> dia >> mes >> anyo;
    switch (mes) {
        case 4: case 6: case 9: case 11:
            diasMes = 30; break;
        case 2:
            diasMes = 28; break;
        default:
            diasMes = 31; break;
    }
    if (dia < diasMes) {            // caso 1: no se acaba el mes
        dia = dia + 1;
    } else if (mes < 12) {          // caso 2: último día de un mes que no es diciembre
        dia = 1;
        mes = mes + 1;
    } else {                        // caso 3: 31 de diciembre
        dia = 1;
        mes = 1;
        anyo = anyo + 1;
    }
    cout << dia << " " << mes << " " << anyo << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Fecha (día mes año): 31 12 2010
1 1 2011
```

Los tres casos cubren todo el dominio (fechas válidas) y no se solapan. En la práctica 4 se reescribe con un `struct Fecha`.

3.3 Trabajo personal

`La compañía de seguros` · **Contrato A, B, experto o rechazado**

Menos de 30 años, buena salud y sin accidentes → contrato A; mala salud *o* algún accidente → llamar a un experto; mala salud *y* accidente → rechazado. Con más de 30 años igual pero contrato B. Codifica el algoritmo del boletín.

EXPLICACIÓN PASO A PASO

Escribir la tabla de verdad primero

Hay tres condiciones booleanas (salud, accidentes, más de 30 años): $2^3 = 8$ combinaciones. Ponerlas en una tabla evita olvidar casos:

Buena salud	Accidentes	Salida
sí	no	Contrato A (≤ 30) o B (> 30)
sí	sí	Llamar a un experto
no	no	Llamar a un experto
no	sí	Rechazado

Cada fila vale para las dos edades, salvo la primera, que se parte en dos. Agrupando por salida salen las cuatro condiciones del SEGÚN.

Comprobar el análisis de casos

- **¿Cubren todo?** Las cuatro filas son todas las combinaciones de salud y accidentes: sí.
- **¿Se solapan?** Cada combinación cae en una sola fila: no.

En C++, «experto» es «las dos ciertas o las dos falsas»: `(salud && accidentes) || (!salud && !accidentes)`. Cuando el programa llega al tercer `else if`, ya ha descartado las otras tres filas, así que sólo queda «buena salud y sin accidentes» y basta mirar la edad.

La tabla de 8 casos del boletín se agrupa por salida. ¿Se cubren todos? Sí: las cuatro condiciones del SEGÚN abarcan las 8 combinaciones. ¿Se solapan? No: «salud y accidentes» o «ninguna de las dos» → experto; «mala salud y accidente» → rechazado; «buena salud sin accidentes» → A o B según la edad.

C++ · SEGUROS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    bool salud, accidentes, edad;
    char respuesta;

    cout << "¿Tiene el cliente buena salud? S/N ";  cin >> respuesta;
    salud = respuesta == 'S';
    cout << "¿Ha sufrido accidentes? S/N ";         cin >> respuesta;
    accidentes = respuesta == 'S';
    cout << "¿Tiene más de 30 años? S/N ";          cin >> respuesta;
    edad = respuesta == 'S';

    if ((salud && accidentes) || (!salud && !accidentes)) {
        cout << "Llamar a un experto" << endl;
    } else if (!salud && accidentes) {
        cout << "Rechazado" << endl;
    } else if (edad) {              // aquí ya sabemos que salud Y NO accidentes
        cout << "Contrato B" << endl;
    } else {
        cout << "Contrato A" << endl;
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
¿Tiene el cliente buena salud? S/N S
¿Ha sufrido accidentes? S/N N
¿Tiene más de 30 años? S/N N
Contrato A
```

En la cadena `if … else if`, al llegar al tercer caso ya se sabe que salud es verdadera y accidentes falso, así que basta mirar la edad.

`Triángulos` · **Triángulo, equilátero, isósceles y escaleno**

Lee tres reales y asigna true o false a Triangulo (cada lado menor que la suma de los otros dos), Equilatero, Isosceles (al menos dos iguales) y Escaleno (ninguno igual).

EXPLICACIÓN PASO A PASO

Cada propiedad es una expresión booleana

No hace falta ningún `if` : una comparación como `a + b > c` ya **es** un valor verdadero o falso, y se puede guardar directamente en una variable `bool` .

- **Triángulo** (desigualdad triangular): cada lado tiene que ser menor que la suma de los otros dos. Si uno fuera igual o mayor, los otros dos no llegarían a juntarse. Hacen falta las **tres** comprobaciones, unidas con Y.
- **Equilátero**: triángulo Y a = b Y b = c (si a = b y b = c, también a = c).
- **Isósceles**: triángulo Y al menos un par igual: (a = b) O (b = c) O (a = c).
- **Escaleno**: triángulo Y los tres pares distintos.

Las tres últimas llevan `triangulo &&` delante: con lados 1, 1, 5 no hay triángulo, y no tendría sentido decir que es isósceles.

Comprobación con 3, 4, 4: 3 + 4 > 4, 3 + 4 > 4, 4 + 4 > 3 → triángulo. Hay un par igual (b = c) → isósceles; no los tres → no equilátero; no todos distintos → no escaleno.

> `cout << boolalpha;` hace que los `bool` se escriban como `true` / `false` ; sin él salen 1/0.

No hace falta ningún `if` : cada variable booleana es el resultado de una expresión booleana. Las tres últimas exigen además `triangulo` .

NOTACIÓN SP

```
Leer (a, b, c);
triangulo ← (a + b > c) Y (a + c > b) Y (b + c > a);
equilatero ← triangulo Y (a = b) Y (b = c);
isosceles ← triangulo Y ((a = b) O (b = c) O (a = c));
escaleno ← triangulo Y (a ≠ b) Y (b ≠ c) Y (a ≠ c)
```

C++ · TRIANGULOS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    float a, b, c;
    bool triangulo, equilatero, isosceles, escaleno;

    cout << "Tres longitudes: ";
    cin >> a >> b >> c;
    triangulo = (a + b > c) && (a + c > b) && (b + c > a);
    equilatero = triangulo && (a == b) && (b == c);
    isosceles = triangulo && ((a == b) || (b == c) || (a == c));
    escaleno = triangulo && (a != b) && (b != c) && (a != c);

    cout << boolalpha;   // escribe true / false en lugar de 1 / 0
    cout << "Triángulo: " << triangulo << endl;
    cout << "Equilátero: " << equilatero << endl;
    cout << "Isósceles: " << isosceles << endl;
    cout << "Escaleno: " << escaleno << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Tres longitudes: 3 4 4
Triángulo: true
Equilátero: false
Isósceles: true
Escaleno: false
```

Según la definición del boletín («al menos dos iguales») un equilátero también es isósceles. `boolalpha` hace que `cout` escriba true/false en vez de 1/0. (Al comparar lados reales con `==` vale lo dicho en la práctica 2 sobre la precisión; con valores tecleados y sin cálculos previos no hay problema.)

`Codificación de caracteres` · **Siguiente letra o dígito, con vuelta**

Lee un carácter y escribe el codificado: si es letra o dígito, el siguiente de su juego (Z → A, z → a, 9 → 0); cualquier otro, un asterisco.

EXPLICACIÓN PASO A PASO

Cómo se obtiene «el siguiente carácter»

Cada carácter es un número en la tabla ASCII y las letras y dígitos están **seguidos**: 'A' = 65, 'B' = 66... Por eso el siguiente de c es `char(int(c) + 1)` : se pasa a número, se suma 1 y se vuelve a carácter. Es el **Sucesor** de la notación.

Los casos, y por qué importa el orden

1. **Los de vuelta**: 'Z' → 'A', 'z' → 'a', '9' → '0'. El sucesor de 'Z' no es 'A' (es '['), así que hay que tratarlos aparte.
2. **El caso general**: letras o dígitos que no son los últimos: 'A'..'Y', 'a'..'y', '0'..'8'. Aquí vale el sucesor.
3. **Todo lo demás** (blancos, signos, tildes) → '*'.

Si el caso general usara 'A'..'Z', la 'Z' entraría en él y saldría '['. Hay dos formas de evitarlo: poner los casos especiales **antes** (como hace la cadena `if … else if`) o recortar los rangos para que **no se solapen** (como hace aquí, con 'Y', 'y' y '8'). El programa hace las dos cosas.

Comprobación: 'g' → 'h'; '5' → '6'; 'Z' → 'A'; '!' → '*'.

Casos: los tres «de vuelta» (Z, z, 9), el caso general de letra o dígito (Sucesor = `char (int (c) + 1)`) y el resto. Los casos especiales van antes para que no los recoja el general (o se excluyen con rangos `'A'..'Y'` , como aquí, para que no se solapen).

C++ · CODIFICA.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    char c, cod;

    cout << "Carácter: ";
    cin >> c;
    if (c == 'Z') {
        cod = 'A';
    } else if (c == 'z') {
        cod = 'a';
    } else if (c == '9') {
        cod = '0';
    } else if (((c ⩾ 'A') && (c ⩽ 'Y')) || ((c ⩾ 'a') && (c ⩽ 'y')) || ((c ⩾ '0') && (c ⩽ '8'))) {
        cod = char(int(c) + 1);     // Sucesor (c)
    } else {
        cod = '*';
    }
    cout << "Codificado: " << cod << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Carácter: Z
Codificado: A
```

`Calificaciones` · **Nota ponderada en formato alfabético**

Con las tres notas ponderadas al 50 %, 30 % y 20 %, escribe Suspenso (< 5), Aprobado [5, 7), Notable [7, 9), Sobresaliente [9, 10) o Matrícula (10).

EXPLICACIÓN PASO A PASO

Los intervalos y cómo se recorren

Nota	Calificación
[0, 5)	Suspenso
[5, 7)	Aprobado
[7, 9)	Notable
[9, 10)	Sobresaliente
10	Matrícula

El corchete [incluye el extremo y el paréntesis) lo excluye: 7 es Notable, no Aprobado. Encadenando `else if` sólo hace falta comparar con el **límite superior**: si llegamos a `else if (nota < 7)` es porque ya no era < 5, así que la nota está en [5, 7).

La matrícula y los reales

La nota sale de una cuenta con decimales (10·0.5 + 10·0.3 + 10·0.2) y en un `float` el resultado puede ser 9.9999999 en vez de 10. Por eso no se compara `nota == 10`, sino «está a menos de EPS de 10»: `fabs (nota - 10) <= EPS` (aquí se usa la negación, `> EPS`, para el Sobresaliente). `fabs` es el valor absoluto de `<cmath>`.

Comprobación: 6, 8, 5 → 6.4 → Aprobado. 10, 10, 10 → 10 → Matrícula. 9, 9, 9 → 9 → Sobresaliente.

Partición guiada por el resultado. Encadenando `else if` sólo hace falta comparar con el límite superior de cada intervalo. Para la matrícula se compara con 10 usando una tolerancia, por la precisión de los reales.

C++ · CALIF.CPP

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main ( ) {
    const float EPS = 0.0001;   // tolerancia para comparar reales
    float examen, trabajo, test, nota;

    cout << "Nota del examen, del trabajo y del test: ";
    cin >> examen >> trabajo >> test;
    nota = examen * 0.5 + trabajo * 0.3 + test * 0.2;

    if (nota < 5) {
        cout << "Suspenso" << endl;
    } else if (nota < 7) {
        cout << "Aprobado" << endl;
    } else if (nota < 9) {
        cout << "Notable" << endl;
    } else if (fabs (nota - 10) > EPS) {   // 9 <= nota < 10
        cout << "Sobresaliente" << endl;
    } else {
        cout << "Matrícula" << endl;
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Nota del examen, del trabajo y del test: 10 10 10
Matrícula
```

PRÁCTICA 4

Productos de tipos y subprogramas

struct, acciones (void) con parámetros por valor y por referencia, funciones y léxico local.

4.1 struct, acciones y funciones

NOTACIÓN

```
Punto = TIPO < abscisa, ordenada : Real >;
Cuadrado = TIPO < p1, p2, p3, p4 : Punto >;
Intercambio : acción (dato-resultado a, b : Real);
Mayor : función (a, b : Real) → Real;
```

C++

```
struct Punto {
    float abscisa, ordenada;
};
struct Cuadrado {
    Punto p1, p2, p3, p4;
};
void Intercambio (float &a, float &b);
float Mayor (float a, float b);
```

- Acceso a campos con punto, encadenable: `c.p3.abscisa = c.p3.abscisa * 2;` (el boletín escribe «abcisa» en la definición y «abscisa» en el uso: tienen que coincidir).
- Faltan los `;` tras las llaves de cada `struct` en el boletín: `struct Punto { … };`.
- **Acción** = `void` + nombre + parámetros. Parámetro **dato** = por **valor** (`float a`); **dato–resultado** y **resultado** = por **referencia** (`float &a`).
- **Función** = tipo del resultado + nombre + parámetros por valor; devuelve con `return`. Puede devolver escalares o structs (un Punto, una Fecha…).
- Los tipos que usan varias funciones se declaran en el **léxico global**, antes de ellas.

EXPLICADO DESPACIO

¿Acción o función?

Pregunta	Si la respuesta es…	Usa
¿Cuántos resultados produce?	exactamente uno	**función**: `tipo Nombre (…) { … return valor; }`
	ninguno, o más de uno	**acción**: `void Nombre (…, tipo &resultado)`

¿Por valor o por referencia?

- **Dato** (sólo se lee) → por **valor**: `int x`. La función recibe una copia y lo que haga con ella no sale fuera.
- **Resultado** (sólo se escribe) o **dato–resultado** (se lee y se modifica) → por **referencia**: `int &x`. La función trabaja sobre la variable del que llama.

Imagina el paso por valor como dar una **fotocopia** (la pueden tachar, tu original no cambia) y por referencia como dar **el original**.

Cómo se ejecuta una llamada

1. Se evalúan los argumentos (parámetros reales).
2. Se emparejan con los parámetros formales **por posición**: el primero con el primero.
3. Los de valor se copian; los de referencia pasan a ser «otro nombre» de la variable.
4. Se ejecuta el cuerpo. Al acabar desaparecen las variables locales y las copias.
5. Si es una función, la llamada «vale» lo que se devolvió con `return`.

Pre y postcondición: la **pre** dice qué tiene que cumplirse al llamar (divisor > 0) y la **post** qué garantiza la función al terminar. Son el «contrato» de cada subprograma.

4.2 Ejercicios de clase

Problema 1 · **Fecha del día siguiente con struct**

Reescribe «la fecha del día siguiente» definiendo un tipo Fecha con el producto de tipos.

EXPLICACIÓN PASO A PASO

Qué cambia respecto a la práctica 3

El algoritmo es el mismo (tres casos), pero ahora los tres números viajan juntos en un **tipo nuevo**:

```
struct Fecha {
    int dia, mes, anyo;
};
```

Una variable `Fecha f;` tiene tres «campos» y se accede a cada uno con un punto: `f.dia` , `f.mes` , `f.anyo` . Es el **producto de tipos** de la notación.

Por qué dos funciones

1. `DiasMes (mes)` devuelve un único valor (cuántos días tiene ese mes): es una **función**.
2. `DiaSiguiente (f)` recibe una fecha y **devuelve otra**: también es una función, porque en C++ una función puede devolver un `struct` entero.
3. `main` sólo lee, llama y escribe: se lee como un resumen del problema.

Separar en subprogramas hace que cada pieza sea pequeña y se pueda probar sola: si DiasMes(2) da 28 y DiasMes(4) da 30, esa parte ya está bien.

> El `struct` se declara **fuera** de `main` (en el léxico global) para que las funciones, que también están fuera, puedan usarlo.

Además del tipo, se aprovecha para separar dos subprogramas: la función `DiasMes` y la función `DiaSiguiente` , que recibe una Fecha y **devuelve** otra (las funciones pueden devolver structs).

C++ · FECHA.CPP

```cpp
#include <iostream>
using namespace std;

struct Fecha {
    int dia, mes, anyo;
};

/* Pre: 1 ⩽ mes ⩽ 12.  Post: días del mes en un año no bisiesto */
int DiasMes (int mes)
{
    int d;
    switch (mes) {
        case 4: case 6: case 9: case 11: d = 30; break;
        case 2:                          d = 28; break;
        default:                         d = 31; break;
    }
    return d;
}

/* Pre: f es una fecha válida de un año no bisiesto.  Post: devuelve la fecha del día siguiente */
Fecha DiaSiguiente (Fecha f)
{
    if (f.dia < DiasMes (f.mes)) {
        f.dia = f.dia + 1;
    } else if (f.mes < 12) {
        f.dia = 1;
        f.mes = f.mes + 1;
    } else {
        f.dia = 1;
        f.mes = 1;
        f.anyo = f.anyo + 1;
    }
    return f;
}

int main ( ) {
    Fecha hoy, manyana;

    cout << "Fecha (día mes año): ";
    cin >> hoy.dia >> hoy.mes >> hoy.anyo;
    manyana = DiaSiguiente (hoy);
    cout << manyana.dia << " " << manyana.mes << " " << manyana.anyo << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Fecha (día mes año): 28 2 2023
1 3 2023
```

`Problema 2` · **Identificar elementos, predecir la salida y modificarla**

En el listado con `BinDec` e `Intercambio`: identifica acciones, parámetros dato y dato–resultado, formales y reales; ponles pre y postcondición; predice la salida sin ejecutar y modifícalo para que escriba «CBA», 13 y 0.

EXPLICACIÓN PASO A PASO

Valor frente a referencia, con este listado

- **Por valor** (`int b1`): la función recibe una **copia**. Si la cambia, el original no se entera. Es un parámetro **dato**.
- **Por referencia** (`int& dec` , con `&`): la función trabaja sobre la **misma variable** del que llama. Lo que escriba ahí se queda. Es un parámetro **resultado** (si sólo escribe) o **dato–resultado** (si lee y escribe).

En `BinDec` , b1…b4 sólo se leen → valor; dec sólo se escribe → referencia (resultado). En `Intercambio` , a y b se leen y se escriben → referencia (dato–resultado).

Predecir la salida: traza a mano

1. x = 'A', y = 'C', z = 'B'.
2. `Intercambio (x, y)` → x = 'C', y = 'A'.
3. `Intercambio (y, z)` → y = 'B', z = 'A'. Ahora x y z = **C B A**.
4. `BinDec (b2, b1, b2, b2, decimal1)` con b1 = 0, b2 = 1: los formales reciben 1, 0, 1, 1 → dec = 1·1 + 0·2 + 1·4 + 1·8 = **13**.
5. `BinDec (0, b1, b1, b1, decimal2)` → todos 0 → **0**.

Formales son los nombres de la cabecera de la función; **reales** (argumentos) son lo que se pone en cada llamada. Se emparejan **por posición**, no por nombre: el primer real va al primer formal.

Elemento	En el listado
Acciones	`BinDec` e `Intercambio`
Parámetros dato (valor)	`b1, b2, b3, b4` de BinDec
Parámetros dato–resultado / resultado (referencia)	`a, b` de Intercambio (dato–resultado); `dec` de BinDec (resultado)
Parámetros formales	`b1, b2, b3, b4, dec` y `a, b`
Parámetros reales (argumentos)	`(x, y)` , `(y, z)` , `(b1, b2, b1, b2, decimal1)` , `(0, 1, b1, b1, decimal2)`

Pre/post: `BinDec` — Pre {b1…b4 ∈ {0, 1}}, Post {dec = b1 + 2·b2 + 4·b3 + 8·b4}; `Intercambio` — Pre {a = A, b = B}, Post {a = B, b = A}. Ojo: los parámetros `b1, b2` de BinDec **enmascaran** a las variables `b1, b2` de main (máxima localidad): son otras variables.

Predicción

1. (x, y, z) = ('O', 'P', 'Q') → Intercambio (x, y) → ('P', 'O', 'Q') → Intercambio (y, z) → ('P', 'Q', 'O'): «**PQO**».
2. BinDec (0, 1, 0, 1): 0 + 2 + 0 + 8 = **10**.
3. BinDec (0, 1, 0, 0): 0 + 2 + 0 + 0 = **2**.

Modificación

Las dos llamadas a Intercambio convierten (a, b, c) en (b, c, a); para obtener (C, B, A) hay que empezar con x = 'A', y = 'C', z = 'B'. 13 = 1101_2 → dec = 1·1 + 0·2 + 1·4 + 1·8: argumentos (1, 0, 1, 1) = `(b2, b1, b2, b2)` . Y 0 con todo ceros. (Otra opción válida: dejar 'A', 'B', 'C' y llamar sólo a `Intercambio (x, z)` .)

C++ · ACCIONES.CPP

```cpp
#include <iostream>
using namespace std;

/* Pre: b1, b2, b3, b4 valen 0 o 1.  Post: dec = b1 + 2·b2 + 4·b3 + 8·b4 */
void BinDec (int b1, int b2, int b3, int b4, int& dec)
{
    dec = b1*1 + b2*2 + b3*4 + b4*8;
}

/* Pre: a = A, b = B.  Post: a = B, b = A */
void Intercambio (char& a, char& b)
{
    char aux;
    aux = a;
    a = b;
    b = aux;
}

int main( ) {
    int b1, b2, decimal1, decimal2;
    char x, y, z;
    b1 = 0;
    b2 = 1;
    x = 'A';                        // antes 'O'
    y = 'C';                        // antes 'P'
    z = 'B';                        // antes 'Q'
    Intercambio (x, y);             // x='C', y='A', z='B'
    Intercambio (y, z);             // x='C', y='B', z='A'
    BinDec (b2, b1, b2, b2, decimal1);   // 1 + 0 + 4 + 8 = 13
    BinDec (0, b1, b1, b1, decimal2);    // 0
    cout << "El valor de las variables x, y, z es: " << x << y << z << endl;
    cout << "El valor del número 1 es: " << decimal1 << endl;
    cout << "El valor del número 2 es: " << decimal2 << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
El valor de las variables x, y, z es: CBA
El valor del número 1 es: 13
El valor del número 2 es: 0
```

Problema 3 · **Acción de división entera y desglose**

Diseña una acción que reciba dividendo y divisor y devuelva cociente y resto; úsala para el desglose de dinero.

EXPLICACIÓN PASO A PASO

Por qué es una acción y no una función

Una función devuelve **un** valor. La división entera produce **dos** (cociente y resto), así que se hace una acción con dos parámetros **resultado** por referencia: `int &cociente, int &resto`.

```
dividendo = divisor · cociente + resto, con 0 ≤ resto < divisor
```

Esa es la postcondición: cualquier pareja (cociente, resto) que la cumpla es la correcta, y sólo hay una.

El detalle de `DivisionEntera (resto, 100, cien, resto)`

1. El primer `resto` va a `dividendo`, que es **por valor**: se copia en el momento de la llamada (por ejemplo, 289).
2. El último `resto` va a `resto`, que es **por referencia**: es la misma variable de main.
3. Dentro se calcula cociente = 289 / 100 = 2 y resto = 289 % 100 = 89 usando la **copia**, así que cambiar el resto de main no estropea la cuenta.

Si `dividendo` también fuera por referencia, al escribir el resto se cambiaría el dividendo a mitad de cálculo. Aquí funciona porque primero se calcula el cociente y luego el resto, pero es frágil: por eso conviene pasar los datos por valor.

Comprobación: 1789 → 3, 2, 1, 1, 1, 1 y 4 en monedas, igual que en la práctica 2.

NOTACIÓN SP

```
DivisionEntera : acción (dato dividendo, divisor : Entero;
                         resultado cociente, resto : Entero)
  PRE { dividendo ≥ 0, divisor > 0 }
  POST { dividendo = divisor · cociente + resto, 0 ≤ resto < divisor }
  ALGORITMO
    cociente ← dividendo DIV divisor;
    resto ← dividendo MOD divisor
  FIN;
…
DivisionEntera (cantidad, 500, quinientos, resto);
DivisionEntera (resto, 100, cien, resto);   // resto es dato y resultado a la vez
…
```

C++ · DIVISION.CPP

```cpp
#include <iostream>
using namespace std;

/* Pre: dividendo ≥ 0, divisor > 0
   Post: dividendo = divisor * cociente + resto, 0 ≤ resto < divisor */
void DivisionEntera (int dividendo, int divisor, int &cociente, int &resto)
{
    cociente = dividendo / divisor;
    resto = dividendo % divisor;
}

int main ( ) {
    int cantidad, resto;
    int quinientos, cien, cincuenta, veinte, diez, cinco;

    cout << "Cantidad en euros: ";
    cin >> cantidad;
    DivisionEntera (cantidad, 500, quinientos, resto);
    DivisionEntera (resto, 100, cien, resto);
    DivisionEntera (resto, 50, cincuenta, resto);
    DivisionEntera (resto, 20, veinte, resto);
    DivisionEntera (resto, 10, diez, resto);
    DivisionEntera (resto, 5, cinco, resto);
    cout << quinientos << " de 500, " << cien << " de 100, " << cincuenta << " de 50, "
         << veinte << " de 20, " << diez << " de 10, " << cinco << " de 5 y "
         << resto << " en monedas" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Cantidad en euros: 1789
3 de 500, 2 de 100, 1 de 50, 1 de 20, 1 de 10, 1 de 5 y 4 en monedas
```

En `DivisionEntera (resto, 100, cien, resto)` el primer argumento se copia (valor) antes de que el último (referencia) lo modifique, así que funciona.

Problema 4 · **Max3 por composición funcional**

Con la función `Max2` dada, construye `Max3` mediante composición funcional.

EXPLICACIÓN PASO A PASO

Composición funcional

El resultado de una función es un **valor**, y un valor se puede usar donde haga falta un dato, incluido como argumento de otra función. Max3 sale de aplicar Max2 dos veces:

```
Max3 (a, b, c) = Max2 (a, Max2 (b, c))
```

1. Primero se evalúa lo de dentro: `Max2 (b, c)` da el mayor de b y c.
2. Después se compara ese valor con a: `Max2 (a, …)` .

Traza con 4, 9, 7: Max2 (9, 7) = 9; Max2 (4, 9) = **9**. Con 8, 2, 5: Max2 (2, 5) = 5; Max2 (8, 5) = **8**.

Ventaja: no hay que escribir ningún `if` nuevo; se reutiliza lo que ya funciona. Es la misma idea que en matemáticas: max(a, b, c) = max(a, max(b, c)).

El resultado de una función es un valor que puede ser argumento de otra: `Max3 = Max2 (a, Max2 (b, c))` .

C++ · MAX3.CPP

```cpp
#include <iostream>
using namespace std;

int Max2 (int a, int b)
{
    int mayor;
    if (a > b) {
        mayor = a; }
    else {
        mayor = b; }
    return mayor;
}

/* Post: devuelve el mayor de a, b y c (composición funcional) */
int Max3 (int a, int b, int c)
{
    return Max2 (a, Max2 (b, c));
}

int main ( ) {
    int x, y, z;
    cout << "Tres enteros: ";
    cin >> x >> y >> z;
    cout << "El máximo es " << Max3 (x, y, z) << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Tres enteros: 6 3 5
El máximo es 6
```

`Problema 5` · **Perímetro de un triángulo**

Identifica en el listado el tipo, la función, las variables, los parámetros y las llamadas. Después define Punto y Triangulo, una acción que lea un triángulo (parámetro resultado), una función Distancia y un algoritmo que calcule el perímetro.

EXPLICACIÓN PASO A PASO

La fórmula de la distancia, de dónde sale

Entre dos puntos P(x1, y1) y Q(x2, y2) se puede dibujar un triángulo rectángulo con catetos |x1 − x2| (horizontal) y |y1 − y2| (vertical). La distancia es la **hipotenusa**, y por Pitágoras:

```
d = √( (x1 - x2)² + (y1 - y2)² )
```

Al elevar al cuadrado el signo da igual, por eso no hace falta el valor absoluto. En C++: `sqrt` de `<cmath>` y una función `sqr (x)` que devuelve x·x.

Del problema a los subprogramas

1. **Tipos:** `Punto` (abscisa, ordenada) y `Triangulo` (tres puntos a, b, c). Un struct puede tener campos que son otros structs: `t.a.abscisa` .
2. **LeerTriangulo (Triangulo &t):** el triángulo es un resultado → referencia. Reutiliza `LeerPunto` tres veces.
3. **Distancia (p, q):** un valor → función.
4. **Perímetro:** suma de los tres lados: d(A, B) + d(B, C) + d(C, A).

Comprobación con A(0, 0), B(3, 0), C(0, 4): d(A, B) = √9 = 3; d(B, C) = √(9 + 16) = 5; d(C, A) = √16 = 4; perímetro **12**.

EL FALLO DEL LISTADO ORIGINAL

El `struct Punto` estaba declarado **dentro** de main. Un tipo declarado dentro de una función sólo existe ahí, así que las funciones de fuera no pueden usarlo. Hay que sacarlo al léxico global, antes de las funciones.

En el listado: tipo `Punto` ; función `sqr` (parámetro formal `x`); variables `p1, p2` (Punto) y `distancia` ; llamadas `sqr (p1.abscisa - p2.abscisa)` y `sqr (p1.ordenada - p2.ordenada)` (esas expresiones son los parámetros reales), además de `sqrt` de `<cmath>` . Problema: `struct Punto` está **dentro de main**, así que no se puede usar en funciones definidas fuera: hay que sacarlo al léxico global.

Fórmula: `d = √((x1 - x2)² + (y1 - y2)²)` . Perímetro = d(A, B) + d(B, C) + d(C, A).

C++ · PERIMETRO.CPP

```cpp
#include <iostream>
#include <cmath>
using namespace std;

struct Punto {
    float abscisa, ordenada;
};
struct Triangulo {
    Punto a, b, c;      // vértices
};

/* Post: devuelve el cuadrado de x */
float sqr (float x)
{
    return x * x;
}

/* Post: devuelve la distancia entre p y q */
float Distancia (Punto p, Punto q)
{
    return sqrt (sqr (p.abscisa - q.abscisa) + sqr (p.ordenada - q.ordenada));
}

/* Post: lee un punto de la entrada estándar */
void LeerPunto (Punto &p)
{
    cout << "  abscisa: ";   cin >> p.abscisa;
    cout << "  ordenada: ";  cin >> p.ordenada;
}

/* Post: t contiene el triángulo leído de la entrada (parámetro resultado) */
void LeerTriangulo (Triangulo &t)
{
    cout << "Vértice A" << endl;  LeerPunto (t.a);
    cout << "Vértice B" << endl;  LeerPunto (t.b);
    cout << "Vértice C" << endl;  LeerPunto (t.c);
}

float Perimetro (Triangulo t)
{
    return Distancia (t.a, t.b) + Distancia (t.b, t.c) + Distancia (t.c, t.a);
}

int main ( ) {
    Triangulo t;
    LeerTriangulo (t);
    cout << "El perímetro es " << Perimetro (t) << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Vértice A
  abscisa: 0
  ordenada: 0
Vértice B
  abscisa: 3
  ordenada: 0
Vértice C
  abscisa: 0
  ordenada: 4
El perímetro es 12
```

Triángulo rectángulo 3-4-5: 3 + 5 + 4 = 12.

4.3 Trabajo personal

Problema 1 · El programa lleno de errores

Área y perímetro de un rectángulo y área y diámetro de un círculo. Encuentra sobre el papel los errores sintácticos, semánticos y lógicos y corrígelos.

EXPLICACIÓN PASO A PASO

Los tres tipos de error, explicados

- **Sintáctico**: lo escrito no es C++ válido y el compilador **no** genera el programa. Ejemplos: `cinin`, un `;` que falta, un paréntesis de más, usar `Dato1` y `dato1` como si fueran lo mismo (C++ distingue mayúsculas).
- **Semántico**: está bien escrito pero no tiene sentido según las reglas del lenguaje. Ejemplo: declarar una función `float` que no devuelve nada.
- **Lógico**: compila y se ejecuta, pero el resultado es incorrecto. Ejemplo: pasar `area` y `perimetro` por valor: la función los calcula en sus copias y main nunca ve los resultados.

Cómo buscarlos sobre el papel

1. Para cada subprograma, pregúntate: ¿devuelve **un** valor (función con `return`) o **varios** (acción `void` con parámetros `&`)?
2. Para cada parámetro: ¿se lee (valor) o se escribe (referencia)?
3. Recorre línea a línea mirando `<<` con `cout` y `>>` con `cin`, los `;` y los paréntesis.
4. Comprueba que cada nombre usado está declarado exactamente igual.

Truco para las flechas: el dato va «hacia» la variable en `cin >> x` y «hacia» la pantalla en `cout << x`.

Error	Tipo	Corrección
`arearec` y `areacir` se declaran `float` pero no devuelven nada	Semántico	Calculan dos resultados: son **acciones** → `void`
`area`, `perimetro` pasados por valor en arearec	Lógico	Son resultados: `float &area`, `float &perimetro`
`& area, &diametro` sin tipo	Sintáctico	`float &area, float &diametro`
`cinlen`, `cinin`, `ci`	Sintáctico	`cin`
`cout >>` y `cinin <<`	Sintáctico	`cout <<` y `cin >>`
`)` sobrante en el segundo mensaje y `;` que falta tras `areacir (…)`	Sintáctico	Quitar / añadir
`Dato1` declarado, `dato1` usado; `Resul` declarado, `Resul1` / `resul1` usados	Sintáctico (mayúsculas)	Nombres coherentes
`area` y `perimetro` usados en main sin declarar	Sintáctico	Variables propias de main
Se asigna el resultado de una acción (`Resul1 = arearec (…)`)	Semántico	Llamarla como instrucción
El radio se lee en `dato2` y los resultados del círculo se guardan en las mismas variables que los del rectángulo antes de escribir estos	Lógico	Variables distintas para cada resultado
Se escribe `resul2` como área del círculo y como diámetro	Lógico	Escribir cada valor en su sitio

C++ · CORREGIDO

```cpp
#include <iostream>
using namespace std;

const float PI = 3.1416;

/* Pre: lado1, lado2 > 0.  Post: área y perímetro del rectángulo */
void AreaRec (float lado1, float lado2, float &area, float &perimetro)
{
    area = lado1 * lado2;
    perimetro = 2 * (lado1 + lado2);
}

/* Pre: radio > 0.  Post: área y diámetro del círculo */
void AreaCir (float radio, float &area, float &diametro)
{
    area = radio * radio * PI;
    diametro = 2 * radio;
}

int main( ) {
    float lado1, lado2, radio;        /* datos de entrada */
    float areaR, perimR, areaC, diam; /* datos de salida */

    cout << "Introduce la longitud de un lado: ";
    cin >> lado1;
    cout << "Introduce la longitud del otro lado: ";
    cin >> lado2;
    AreaRec (lado1, lado2, areaR, perimR);
    cout << "Introduce el radio del círculo: ";
    cin >> radio;
    AreaCir (radio, areaC, diam);
    cout << "El área del rectángulo es " << areaR << " y el perímetro " << perimR << endl;
    cout << "El área del círculo es " << areaC << " y el diámetro " << diam << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Introduce la longitud de un lado: 3
Introduce la longitud del otro lado: 4
Introduce el radio del círculo: 1
El área del rectángulo es 12 y el perímetro 14
El área del círculo es 3.1416 y el diámetro 2
```

3 × 4: área 12, perímetro 14. Radio 1: área 3.1416, diámetro 2. Como dice el boletín: corrige sobre el papel; cambiar cosas al azar hasta que compile no hace un programa correcto.

`Problema 2` · **Clasificación energética de una lavadora**

Define el tipo Lavadora (marca, año, peso, motor inverter o convencional, consumo por ciclo en kWh, duración del ciclo en minutos), una acción que la lea (sólo valores escalares), una acción que devuelva el consumo por minuto y la eficiencia relativa, una función con la etiqueta (A: < 0.5 e inverter; B: [0.5, 0.8); C: [0.8, 1.2); D: ≥ 1.2) y el algoritmo principal.

EXPLICACIÓN PASO A PASO

Del enunciado al diseño

1. **El tipo**: un campo por cada dato de la lavadora. El motor sólo puede ser inverter o convencional → un `bool inverter`.
2. **Leer «sólo valores escalares»**: no se puede hacer `cin >> lavadora`; se lee campo a campo. Para el motor se lee un carácter (I/C) y se convierte: `l.inverter = (motor == 'I') || (motor == 'i');`
3. **Dos indicadores** → una acción con dos parámetros por referencia.
4. **Una etiqueta** (un solo carácter) → una función que devuelve `char`.

Las fórmulas

- **Consumo por minuto** = kWh del ciclo / minutos del ciclo. Con 0.6 kWh y 60 min: 0.6 / 60 = **0.01 kWh/min**.
- **Eficiencia relativa** = 1 / consumo por minuto: cuantos menos kWh por minuto, más eficiente. 1 / 0.01 = **100**.

La etiqueta como análisis de casos

A: consumo < 0.5 Y inverter. B: < 0.8. C: < 1.2. D: el resto. En la cadena `else if` cada condición sólo mira el límite superior porque las anteriores ya han descartado los valores menores. La lavadora del ejemplo (0.6 kWh) no es A (0.6 ≥ 0.5) pero sí cumple < 0.8 → **B**.

El hueco del enunciado (consumo < 0.5 sin inverter) cae, con este código, en B: es una decisión que hay que dejar escrita.

El tipo de motor tiene dos valores: un `bool inverter`. La acción de lectura lee un carácter (I/C) y lo convierte. Los indicadores son dos resultados → una acción con dos parámetros por referencia; la etiqueta es un único valor → una función que devuelve `char`.

UN CASO QUE EL ENUNCIADO NO CUBRE

Una lavadora con consumo < 0.5 y motor **convencional** no cumple la A ni entra en el intervalo de la B. Es justo el error de **cobertura** del análisis de casos del tema 1. Aquí se ha decidido darle la B (no puede ser A, y consume menos que el límite de la B); indícalo en tu solución si eliges otra cosa.

C++ · LAVADORA.CPP

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Lavadora {
    string marca;
    int anyo;              // año de fabricación
    float peso;            // kg
    bool inverter;         // true: inverter, false: convencional
    float consumo;         // kWh por ciclo
    int duracion;          // minutos del ciclo estándar
};

/* Post: l contiene la lavadora leída (sólo se leen valores escalares) */
void LeerLavadora (Lavadora &l)
{
    char motor;
    cout << "Marca: ";                          cin >> l.marca;
    cout << "Año de fabricación: ";             cin >> l.anyo;
    cout << "Peso (kg): ";                      cin >> l.peso;
    cout << "Motor (I = inverter, C = convencional): ";  cin >> motor;
    l.inverter = (motor == 'I') || (motor == 'i');
    cout << "Consumo por ciclo (kWh): ";        cin >> l.consumo;
    cout << "Duración del ciclo (min): ";       cin >> l.duracion;
}

/* Pre: l.duracion > 0, l.consumo > 0
   Post: consumoMin = l.consumo / l.duracion, eficiencia = 1 / consumoMin */
void Indicadores (Lavadora l, float &consumoMin, float &eficiencia)
{
    consumoMin = l.consumo / l.duracion;
    eficiencia = 1 / consumoMin;
}

/* Post: etiqueta energética de l ('A', 'B', 'C' o 'D') */
char Etiqueta (Lavadora l)
{
    char e;
    if (l.consumo < 0.5 && l.inverter) {
        e = 'A';
    } else if (l.consumo < 0.8) {      // incluye < 0.5 sin inverter (caso no cubierto por el enunciado)
        e = 'B';
    } else if (l.consumo < 1.2) {
        e = 'C';
    } else {
        e = 'D';
    }
    return e;
}

int main ( ) {
    Lavadora lav;
    float consumoMin, eficiencia;

    LeerLavadora (lav);
    Indicadores (lav, consumoMin, eficiencia);
    cout << "Marca: " << lav.marca << endl;
    cout << "Etiqueta energética: " << Etiqueta (lav) << endl;
    cout << "Consumo por minuto: " << consumoMin << " kWh/min" << endl;
    cout << "Eficiencia relativa: " << eficiencia << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Marca: Bosch
Año de fabricación: 2023
Peso (kg): 70.5
Motor (I = inverter, C = convencional): I
Consumo por ciclo (kWh): 0.6
Duración del ciclo (min): 60
Marca: Bosch
Etiqueta energética: B
Consumo por minuto: 0.01 kWh/min
Eficiencia relativa: 100
```

Con el ejemplo del enunciado (0.6 kWh en 60 min): 0.01 kWh/min y eficiencia 100, etiqueta B.

`Problema 3` · **Autonomía de una bicicleta eléctrica**

Tipo Bicicleta (marca, freno de disco o de llanta, año, peso, autonomía en km, tiempo de carga en minutos), acción de lectura, acción con velocidad de recarga (autonomía / tiempo de carga) y autonomía por kg (autonomía / peso), función con la etiqueta (A ≥ 100; M 60–99; B 30–59; L < 30) y algoritmo principal.

EXPLICACIÓN PASO A PASO

Las fórmulas, mirando las unidades

- **Velocidad de recarga** = autonomía / tiempo de carga → km / min: cuántos kilómetros de batería se recuperan por minuto. 80 / 40 = **2 km/min**.
- **Autonomía por kg** = autonomía / peso → km / kg. 80 / 25 = **3.2 km/kg**.

Como son dos resultados, van en una acción con dos parámetros por referencia (igual que en la lavadora).

La etiqueta sin huecos

El enunciado da intervalos de enteros: A ≥ 100, M 60–99, B 30–59, L < 30. Si la autonomía es real, ¿qué pasa con 99.5 km? No está ni en «60–99» ni en «≥ 100». La solución es comparar sólo con el **límite inferior** en orden descendente:

1. ¿≥ 100? → A.
2. Si no, ¿≥ 60? → M (aquí cae 99.5).
3. Si no, ¿≥ 30? → B.
4. Si no → L.

Así se cubre todo el dominio sin solapes. Con 80 km: no ≥ 100, sí ≥ 60 → **M**.

Misma estructura que la lavadora. Si la autonomía es real, los intervalos «60–99» y «30–59» dejan huecos (¿99.5 km?): comparando sólo con el límite inferior en una cadena de `else if` se cubre todo el dominio sin solapes.

C++ · BICI.CPP

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Bicicleta {
    string marca;
    bool disco;            // true: freno de disco, false: de llanta
    int anyo;
    float peso;            // kg
    float autonomia;       // km
    float tiempoCarga;     // minutos
};

void LeerBicicleta (Bicicleta &b)
{
    char freno;
    cout << "Marca: ";                              cin >> b.marca;
    cout << "Freno (D = disco, L = llanta): ";      cin >> freno;
    b.disco = (freno == 'D') || (freno == 'd');
    cout << "Año de fabricación: ";                 cin >> b.anyo;
    cout << "Peso (kg): ";                          cin >> b.peso;
    cout << "Autonomía (km): ";                     cin >> b.autonomia;
    cout << "Tiempo de carga (min): ";              cin >> b.tiempoCarga;
}

/* Pre: b.tiempoCarga > 0, b.peso > 0 */
void Indicadores (Bicicleta b, float &velRecarga, float &autonomiaKg)
{
    velRecarga = b.autonomia / b.tiempoCarga;
    autonomiaKg = b.autonomia / b.peso;
}

char Etiqueta (Bicicleta b)
{
    char e;
    if (b.autonomia ⩾ 100) {
        e = 'A';
    } else if (b.autonomia ⩾ 60) {
        e = 'M';
    } else if (b.autonomia ⩾ 30) {
        e = 'B';
    } else {
        e = 'L';
    }
    return e;
}

int main ( ) {
    Bicicleta bici;
    float velRecarga, autonomiaKg;

    LeerBicicleta (bici);
    Indicadores (bici, velRecarga, autonomiaKg);
    cout << "Marca: " << bici.marca << endl;
    cout << "Etiqueta de autonomía: " << Etiqueta (bici) << endl;
    cout << "Velocidad de recarga: " << velRecarga << " km/min" << endl;
    cout << "Autonomía por kg: " << autonomiaKg << " km/kg" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Marca: Orbea
Freno (D = disco, L = llanta): D
Año de fabricación: 2024
Peso (kg): 25
Autonomía (km): 80
Tiempo de carga (min): 40
Marca: Orbea
Etiqueta de autonomía: M
Velocidad de recarga: 2 km/min
Autonomía por kg: 3.2 km/kg
```

Ejemplo del enunciado: 80 km, 40 min, 25 kg → 2 km/min y 3.2 km/kg, etiqueta M.

Problema 4 · **Rectángulos: validez y contención**

Tipo Punto y tipo Rectángulo (esquina superior izquierda e inferior derecha); acción de lectura; función que diga si es válido; acción que diga si r2 está contenido en r1 y devuelva la diferencia de áreas; algoritmo principal.

EXPLICACIÓN PASO A PASO

Elegir bien los ejes

Se toma el plano con el eje **y hacia arriba** (como en matemáticas). Entonces «la esquina superior izquierda está por encima y a la izquierda de la inferior derecha» se traduce en dos comparaciones:

```
válido = (supIzq.x < infDer.x) Y (supIzq.y > infDer.y)
```

Área

Ancho = infDer.x − supIzq.x; alto = supIzq.y − infDer.y (las dos restas salen positivas si el rectángulo es válido). **Área = ancho · alto**.

¿Está r2 dentro de r1?

r2 está dentro si no se sale por ninguno de los cuatro lados:

1. Su lado izquierdo está a la derecha del de r1: r2.supIzq.x ≥ r1.supIzq.x.
2. Su lado derecho está a la izquierda del de r1: r2.infDer.x ≤ r1.infDer.x.
3. Su lado de arriba está por debajo del de r1: r2.supIzq.y ≤ r1.supIzq.y.
4. Su lado de abajo está por encima del de r1: r2.infDer.y ≥ r1.infDer.y.

Las cuatro unidas con Y. La **diferencia de áreas** se da como valor absoluto (`fabs`) para que salga positiva sea cual sea el mayor.

Comprobación: r1 = (0, 10)–(10, 0): 10 × 10 = 100. r2 = (2, 8)–(5, 3): 3 × 5 = 15. 2 ≥ 0, 5 ≤ 10, 8 ≤ 10, 3 ≥ 0 → contenido; diferencia **85**.

Se toma el plano con el eje y hacia arriba: «por encima y a la izquierda» es `supIzq.x < infDer.x` y `supIzq.y > infDer.y`. r2 está dentro de r1 si sus dos esquinas están dentro de los límites de r1. La diferencia de áreas es el valor absoluto de la resta (área mayor − menor).

C++ · RECTANGULOS.CPP

```cpp
#include <iostream>
#include <cmath>
using namespace std;

struct Punto {
    float x, y;
};
struct Rectangulo {
    Punto supIzq, infDer;   // esquina superior izquierda e inferior derecha
};

void LeerRectangulo (Rectangulo &r)
{
    cout << "Esquina superior izquierda (x y): ";  cin >> r.supIzq.x >> r.supIzq.y;
    cout << "Esquina inferior derecha (x y): ";    cin >> r.infDer.x >> r.infDer.y;
}

/* Post: Verdadero si supIzq está por encima y a la izquierda de infDer (eje y hacia arriba) */
bool EsValido (Rectangulo r)
{
    return (r.supIzq.x < r.infDer.x) && (r.supIzq.y > r.infDer.y);
}

float Area (Rectangulo r)
{
    return (r.infDer.x - r.supIzq.x) * (r.supIzq.y - r.infDer.y);
}

/* Pre: r1 y r2 válidos
   Post: contenido = r2 está dentro de r1; difArea = área mayor - área menor */
void Relacion (Rectangulo r1, Rectangulo r2, bool &contenido, float &difArea)
{
    contenido = (r2.supIzq.x ⩾ r1.supIzq.x) && (r2.infDer.x ⩽ r1.infDer.x)
             && (r2.supIzq.y ⩽ r1.supIzq.y) && (r2.infDer.y ⩾ r1.infDer.y);
    difArea = fabs (Area (r1) - Area (r2));
}

int main ( ) {
    Rectangulo r1, r2;
    bool contenido;
    float dif;

    cout << "Rectángulo 1" << endl;  LeerRectangulo (r1);
    cout << "Rectángulo 2" << endl;  LeerRectangulo (r2);
    if (EsValido (r1)) { cout << "El rectángulo 1 es válido" << endl; }
    else { cout << "El rectángulo 1 no es válido" << endl; }
    if (EsValido (r2)) { cout << "El rectángulo 2 es válido" << endl; }
    else { cout << "El rectángulo 2 no es válido" << endl; }
    if (EsValido (r1) && EsValido (r2)) {
        Relacion (r1, r2, contenido, dif);
        if (contenido) { cout << "El segundo está contenido en el primero" << endl; }
        else { cout << "El segundo no está contenido en el primero" << endl; }
        cout << "Diferencia de áreas: " << dif << endl;
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Rectángulo 1
Esquina superior izquierda (x y): 0 10
Esquina inferior derecha (x y): 10 0
Rectángulo 2
Esquina superior izquierda (x y): 2 8
Esquina inferior derecha (x y): 5 3
El rectángulo 1 es válido
El rectángulo 2 es válido
El segundo está contenido en el primero
Diferencia de áreas: 85
```

r1 = (0, 10)–(10, 0), área 100; r2 = (2, 8)–(5, 3), área 15: contenido, diferencia 85.

`Problema 5` · **Circunferencias: perímetro e intersección**

Tipos Punto y Circunferencia; acción de lectura; función con el perímetro (2·π·r); acción que diga si dos circunferencias intersecan; algoritmo principal.

EXPLICACIÓN PASO A PASO

Perímetro

La longitud de una circunferencia es **2 · π · r**. Con r = 5: 2 · 3.14159 · 5 = 31.4159. (Ojo: `PI` tiene que ser `float`; si fuera `int` valdría 3.)

Cuándo se cortan dos circunferencias

Todo depende de la distancia **d** entre los centros comparada con los radios:

Situación	Condición
Separadas (no se tocan)	d > r1 + r2
Se cortan o son tangentes	|r1 − r2| ≤ d ≤ r1 + r2
Una dentro de la otra, sin tocarse	d < |r1 − r2|

Intuición: si los centros están más lejos que la suma de los radios, «no llegan»; si están tan cerca que la pequeña cabe entera dentro de la grande sin tocar el borde, tampoco se cortan.

1. d = √(dx² + dy²), con dx y dy las diferencias de coordenadas de los centros (Pitágoras, como en la distancia entre puntos).
2. corta = (d ≤ r1 + r2) Y (d ≥ |r1 − r2|).

Comprobación: centros (0, 0) y (8, 0) → d = 8; radios 5 y 4 → |5 − 4| = 1 ≤ 8 ≤ 9 → **se cortan**.

Dos **circunferencias** (las curvas) tienen algún punto en común cuando la distancia d entre centros cumple `|r1 - r2| ≤ d ≤ r1 + r2`. Si d > r1 + r2 están separadas; si d < |r1 − r2|, una está dentro de la otra sin tocarse. (Si el enunciado se refiriera a los *círculos*, bastaría d ≤ r1 + r2.)

C++ · CIRCUNFERENCIAS.CPP

```cpp
#include <iostream>
#include <cmath>
using namespace std;

const float PI = 3.14159;

struct Punto {
    float x, y;
};
struct Circunferencia {
    Punto centro;
    float radio;
};

void LeerCircunferencia (Circunferencia &c)
{
    cout << "Centro (x y): ";  cin >> c.centro.x >> c.centro.y;
    cout << "Radio: ";         cin >> c.radio;
}

float Perimetro (Circunferencia c)
{
    return 2 * PI * c.radio;
}

/* Post: corta = las dos circunferencias tienen algún punto en común
         ( |r1 - r2| ≤ d ≤ r1 + r2 ) */
void Intersecan (Circunferencia c1, Circunferencia c2, bool &corta)
{
    float dx, dy, d;
    dx = c1.centro.x - c2.centro.x;
    dy = c1.centro.y - c2.centro.y;
    d = sqrt (dx * dx + dy * dy);
    corta = (d ≤ c1.radio + c2.radio) && (d ≥ fabs (c1.radio - c2.radio));
}

int main ( ) {
    Circunferencia c1, c2;
    bool corta;

    cout << "Circunferencia 1" << endl;  LeerCircunferencia (c1);
    cout << "Circunferencia 2" << endl;  LeerCircunferencia (c2);
    cout << "Perímetro de la 1: " << Perimetro (c1) << endl;
    cout << "Perímetro de la 2: " << Perimetro (c2) << endl;
    Intersecan (c1, c2, corta);
    if (corta) { cout << "Las circunferencias intersecan" << endl; }
    else { cout << "Las circunferencias no intersecan" << endl; }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Circunferencia 1
Centro (x y): 0 0
Radio: 5
Circunferencia 2
Centro (x y): 8 0
Radio: 4
Perímetro de la 1: 31.4159
Perímetro de la 2: 25.1327
Las circunferencias intersecan
```

Centros a distancia 8, radios 5 y 4: $1 \le 8 \le 9$ → intersecan.

PRÁCTICA 5

Iteraciones

while, do–while y for frente a MIENTRAS, REPETIR, ITERAR y RECORRIENDO; recorridos, búsquedas, sucesiones y series.

5.1 Las composiciones iterativas en C++

Notación	C++
`MIENTRAS cond HACER S FIN_MIENTRAS`	`while (cond) { S; }`
`REPETIR S HASTA QUE cond`	`do { S; } while (!(cond));`
`ITERAR S1 DETENER cond S2 FIN_ITERAR`	`S1; while (!(cond)) { S2; S1; }`
`i RECORRIENDO [a, b] HACER S`	`for (i = a; i ⩽ b; i = i + 1) { S; }`
`… EN SENTIDO INVERSO`	`for (i = b; i ⩾ a; i = i - 1) { S; }`

C++ no tiene ITERAR. Además de la equivalencia de la tabla, se puede escribir como iteración infinita con parada central:

ITERAR CON BREAK

```
do {
    S1;
    if (cond) break;      // el boletín pone «then»: en C++ no existe
    S2;
} while (true);
```

BREAK SÓLO PARA ESTO

Según el boletín, `break` sólo se puede usar en este tipo de iteraciones. Usarlo para salir de cualquier otro bucle se considera un **grave fallo de concepto** y se penaliza.

Los ejemplos del boletín: contar blancos de una línea leyendo con `cin.get (c)` hasta `'\n'` (while), contar los términos de la serie armónica hasta superar un límite (do–while) y las potencias de 2 en [x, y] calculando cada una a partir de la anterior (for).

EXPLICADO DESPACIO

Elegir la composición iterativa

Situación	Notación	C++
Sé cuántas vueltas habrá antes de empezar	RECORRIENDO	`for`
Puede que no haya que entrar ninguna vez	MIENTRAS	`while`
Hay que hacerlo al menos una vez (pedir un dato hasta que sea válido)	REPETIR … HASTA	`do … while` (con la condición **negada**)
La salida está en medio del cuerpo	ITERAR … DETENER	`S1; while (!cond) { S2; S1; }`

REPETIR … HASTA Y DO … WHILE VAN AL REVÉS

En REPETIR la condición es de **parada** («hasta que nota sea válida»); en `do … while` es de **continuación** («mientras nota no sea válida»). Al traducir hay que negarla: `do { … } while (!(nota ⩾ 0 && nota ⩽ 10));`

Las cuatro preguntas para diseñar cualquier bucle

1. **¿Qué se cumple en cada vuelta?** (el invariante: «suma es la suma de lo leído»).
2. **¿Cómo empiezo?** Valores iniciales que hagan cierto el invariante antes de la primera vuelta (suma = 0, producto = 1, contador = 0).
3. **¿Qué hago en cada vuelta?** El tratamiento que mantiene el invariante y el avance que acerca al final.
4. **¿Cuándo paro?** La condición. Comprueba que el avance la hace falsa en algún momento: si no, el bucle no termina.

5.2 Ejercicios de clase

Problema 1 · Codificar una frase

Como la codificación de un carácter de la práctica 3, pero para una frase completa, leyendo carácter a carácter (como en el ejemplo 1).

EXPLICACIÓN PASO A PASO

El esquema: recorrido de una línea

Una frase es una **secuencia de caracteres** que termina en el salto de línea `'\n'` (la marca de fin). Hay que tratar todos los caracteres: es un **recorrido**, que siempre tiene esta forma:

1. **Comenzar**: leer el primer carácter antes del bucle (`cin.get(c);`).
2. **Mientras** no sea la marca (`c ≠ '\n'`):
3. **tratar** el carácter actual: escribir `Codificar (c)` ;
4. **avanzar**: leer el siguiente (`cin.get(c);`) al final del cuerpo.

La lectura va **dos veces**: una antes del bucle y otra al final del cuerpo. Así la condición siempre se comprueba sobre un carácter recién leído y la marca nunca se trata como si fuera un dato.

Por qué `cin.get` y no `cin >>`

`cin >> c` se salta los blancos y los saltos de línea: nunca leería el `'\n'` , el bucle no terminaría y además los espacios no se codificarían como '*'. `cin.get(c)` lee **cualquier** carácter, también blancos y `'\n'` .

Comprobación: «Hola Zoe» → H→I, o→p, l→m, a→b, ' '→*, Z→A, o→p, e→f → «Ipmb*Apf».

Se mete la codificación en una función `Codificar` y se recorre la línea: `cin.get` antes del bucle (Comenzar) y al final del cuerpo (Avanzar). Con `cin >> c` no funcionaría: se salta los blancos y el `'\n'` , así que el bucle no acabaría al final de la línea.

C++ · FRASE.CPP

```cpp
#include <iostream>
using namespace std;

/* Post: devuelve el carácter codificado de c (siguiente del juego, con vuelta; '*' si no es letra ni dígito) */
char Codificar (char c)
{
    char cod;
    if (c == 'Z') { cod = 'A'; }
    else if (c == 'z') { cod = 'a'; }
    else if (c == '9') { cod = '0'; }
    else if (((c ≥ 'A') && (c ≤ 'Y')) || ((c ≥ 'a') && (c ≤ 'y')) || ((c ≥ '0') && (c ≤ '8'))) {
        cod = char(int(c) + 1);
    } else { cod = '*'; }
    return cod;
}

int main( ) {
    char c;    // carácter actual

    cout << "Escribe una frase: ";
    cin.get(c);                          // Comenzar
    while (c ≠ '\n') {                   // fin de línea
        cout << Codificar (c);           // tratamiento
        cin.get(c);                      // Avanzar
    }
    cout << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Escribe una frase: Hola Zoe, son las 9!
Ipmb*Apf**tpo*mbt*0*
```

Problema 2 · **Cociente y resto por restas sucesivas**

Dados x ≥ 0 e y > 0, calcula c = x DIV y y r = x MOD y restando. Al acabar: x = y·c + r con 0 ≤ r < y. ¿Valores iniciales? ¿Actualización? ¿Condición de fin?

EXPLICACIÓN PASO A PASO

La idea: dividir es restar muchas veces

17 entre 5: le quito 5 → 12, otra vez → 7, otra → 2. Ya no cabe otro 5. He restado **3 veces** (el cociente) y quedan 2 (el resto).

El invariante guía el diseño

La propiedad que se mantiene en cada vuelta es **x = y·c + r** (el dividendo es «lo que llevo restado» más «lo que queda»).

1. **Inicialización**: hay que hacer que el invariante sea cierto antes de empezar. Con c = 0 y r = x: x = y·0 + x ✓.
2. **Cuerpo**: restar y una vez: r ← r − y y apuntarlo: c ← c + 1. Sigue valiendo, porque y·(c + 1) + (r − y) = y·c + r.
3. **Condición de continuación**: seguir mientras quepa otro y: **r ≥ y**.
4. **Al salir**: r < y (por la condición) y r ≥ 0 (porque sólo se resta cuando r ≥ y). Junto con el invariante, eso es exactamente la postcondición x = y·c + r, 0 ≤ r < y.

Vuelta	c	r	¿r ≥ 5?
inicio	0	17	sí
1	1	12	sí
2	2	7	sí
3	3	2	no → fin

¿Por qué `while`? Porque no se sabe de antemano cuántas vueltas habrá (eso es justo lo que se busca). Con x < y no entra ninguna vez y da c = 0, r = x, que es correcto.

- **Inicial**: c ← 0, r ← x (x = y·0 + x: el invariante se cumple).
- **Cuerpo**: r ← r − y; c ← c + 1 (sigue cumpliéndose x = y·c + r).
- **Continuación**: r ≥ y. Al salir r < y y, como sólo se resta cuando r ≥ y, r ≥ 0: postcondición.

C++ · RESTAS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int x, y;       // dividendo ≥ 0 y divisor > 0
    int c, r;       // cociente y resto

    cout << "Dividendo y divisor: ";
    cin >> x >> y;
    c = 0;                        // x = y*0 + x
    r = x;
    while (r ≥ y) {               // Inv: x = y*c + r, r ≥ 0
        r = r - y;
        c = c + 1;
    }
    // r < y: x = y*c + r, 0 ≤ r < y
    cout << "Cociente " << c << ", resto " << r << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Dividendo y divisor: 17 5
Cociente 3, resto 2
```

`Problema 3` · **Cuenta de ahorro**

$a_0 = 125$ y $a_n = a_{n-1} + (0.04/12) \cdot a_{n-1} + 180$. (a) ¿Cuántos meses hay que esperar para tener E euros? (b) ¿Cuánto habrá en el mes M?

EXPLICACIÓN PASO A PASO

La fórmula del saldo

$$a_n = a_{n-1} + (0.04 / 12) \cdot a_{n-1} + 180$$

- a_{n-1}: lo que había el mes anterior.
- $(0.04 / 12) \cdot a_{n-1}$: el interés del mes. Un 4 % **anual** repartido en 12 meses da $0.04/12 \approx 0.00333$ por mes.
- + 180: el ingreso fijo de cada mes.

Para calcular un mes sólo hace falta el anterior, así que basta **una variable** `saldo` que se va actualizando: no hace falta guardar toda la sucesión.

Los primeros meses, a mano

Mes	Cuenta	Saldo
0	inicial	125.00
1	125 + 0.4167 + 180	305.42
2	305.42 + 1.0181 + 180	486.43
3	486.43 + 1.6214 + 180	668.06

Dos preguntas, dos bucles distintos

1. **(a) ¿Cuántos meses para tener E?** No se sabe cuántas vueltas hay: es una **búsqueda** del primer mes con saldo ≥ E → `while (saldo < E)`, con un contador n. Con E = 3000 salen **16** meses (saldo 3084.97).
2. **(b) ¿Cuánto hay en el mes M?** Se sabe que hay que dar exactamente M vueltas: es un **recorrido** → `for (i = 1; i ⩽ M; …)`. Con M = 12: **2330.14 €**.

Regla para elegir: **sé cuántas vueltas → for; paro cuando se cumpla algo → while**. Antes de la parte (b) hay que reiniciar `saldo` a 125, porque la parte (a) lo ha dejado cambiado.

(a) es una **búsqueda** del primer término ≥ E: no se sabe cuántas vueltas hará → `while`. (b) es un **recorrido** de M términos conocidos de antemano → `for`. En los dos sólo hace falta el término anterior, no toda la sucesión.

C++ · AHORRO.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const double INICIAL = 125, INTERES = 0.04, INGRESO = 180;
    double E, saldo;
    int n, M, i;

    cout << "Euros que quieres reunir: ";
    cin >> E;
    cout << "Mes que quieres consultar: ";
    cin >> M;

    // a) número de meses hasta tener al menos E euros
    n = 0;
    saldo = INICIAL;                      // a0
    while (saldo < E) {
        n = n + 1;
        saldo = saldo + (INTERES / 12) * saldo + INGRESO;   // an
    }
    cout << "a) Hay que esperar " << n << " meses (saldo " << saldo << ")" << endl;

    // b) saldo del mes M
    saldo = INICIAL;
    for (i = 1; i ⩽ M; i = i + 1) {
        saldo = saldo + (INTERES / 12) * saldo + INGRESO;
    }
    cout << "b) En el mes " << M << " habrá " << saldo << " euros" << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Euros que quieres reunir: 3000
Mes que quieres consultar: 12
a) Hay que esperar 16 meses (saldo 3084.97)
b) En el mes 12 habrá 2330.14 euros
```

5.3 Trabajo personal

Problema 1 · Los ejemplos 1 y 2 con la otra composición

Resuelve el ejemplo 1 con do–while y el 2 con while. ¿Hacen falta otras composiciones de control para ajustar la semántica?

EXPLICACIÓN PASO A PASO

Qué cambia entre `while` y `do-while`

- `while (cond) { … }` comprueba **antes**: el cuerpo puede ejecutarse 0 veces.
- `do { … } while (cond);` comprueba **después**: el cuerpo se ejecuta al menos 1 vez.

Ejemplo 1 con `do-while` (contar blancos)

Ahora la lectura va dentro, al principio del cuerpo, y la condición se mira al final. El último carácter leído es el `'\n'` y se «trata» (se pregunta si es un blanco) antes de salir. No pasa nada porque no es un blanco: el contador no cambia. Si la tarea fuera «contar caracteres», sí habría que corregirlo.

Ejemplo 2 con `while` (sumar 1 + 1/2 + 1/3 + … hasta pasar de 5)

La condición `suma ⩽ LIMITE` es cierta al principio (suma = 0), así que el `while` entra al menos una vez, igual que el `do-while`: dan el mismo resultado, 83 términos.

> LA REGLA GENERAL (TEMA 3)
> De `while` a `do-while`: `if (cond) do { S } while (cond);` (el `if` cubre el caso de 0 vueltas). De `do-while` a `while`: `S; while (cond) { S }` (se ejecuta una vez fuera). En estos dos ejemplos no ha hecho falta por suerte, no porque siempre dé igual.

C++ · EJ1DOWHILE.CPP

```cpp
#include <iostream>
using namespace std;

int main( ) {
    int cont;   // Contador de blancos
    char c;     // Carácter actual
    cont = 0;
    do {
        cin.get(c);
        if (c == ' ') {
            cont = cont + 1;
        }
    } while (c ≠ '\n');   // el '\n' se «trata», pero no es un blanco: no altera cont
    cout << "El número de blancos es : " << cont << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
a b  c d
El número de blancos es : 4
```

C++ · EJ2WHILE.CPP

```cpp
#include <iostream>
using namespace std;

int main( ) {
    const int LIMITE = 5;
    float suma;
    int cuenta;
    cuenta = 0;
    suma = 0;
    while (suma ⩽ LIMITE) {     // al principio 0 ⩽ LIMITE: entra al menos una vez
        cuenta = cuenta + 1;
        suma = suma + 1.0/cuenta;
    }
    cout << "El número de términos es : " << cuenta << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
El número de términos es : 83
```

No hacen falta en estos dos casos, por suerte: en el 1, el do–while «trata» también el `'\n'`, pero no es un blanco y no altera el contador; en el 2, la condición es cierta al principio (suma = 0 ≤ 5), así que el while entra al menos una vez, igual que el do–while. En general, pasar de while a do–while exige un `if` delante (por si no hay que entrar) y al revés, repetir el cuerpo antes del bucle (tema 3, equivalencias).

Problema 2 · **¿Es primo? (versión mejorada)**

Mejora el algoritmo de clase: los pares no son primos (salvo el 2) y basta buscar divisores hasta la raíz cuadrada.

EXPLICACIÓN PASO A PASO

Por qué basta buscar hasta la raíz

Si $a = d \cdot e$ con $d \le e$, entonces $d \cdot d \le d \cdot e = a$, es decir, $\mathbf{d \le \sqrt{a}}$. Todo número compuesto tiene un divisor que no pasa de su raíz. Si no hay ninguno hasta $\sqrt{a}$, no hay ninguno: es primo.

Por qué sólo impares

Si a es impar, ningún par lo divide. Así que, tras tratar aparte los pares (el 2 es primo; el resto, no), se prueban d = 3, 5, 7, 9… sumando 2.

El algoritmo como búsqueda

1. a < 2 → no primo (el 1 no es primo).
2. a par → primo sólo si a = 2.
3. a impar: d = 3; **mientras** d·d ≤ a **y** d no divida a a → d = d + 2.
4. Al salir hay dos motivos posibles: se pasó la raíz (d·d > a → primo) o se encontró un divisor (no primo). Se distinguen mirando `d * d > a`.

Se compara `d * d ⩽ a` en vez de `d ⩽ sqrt(a)` para trabajar sólo con enteros y evitar errores de redondeo.

Traza con 97 ($\sqrt{97} \approx 9.85$): d = 3 (9 ≤ 97, resto 1), 5 (25, resto 2), 7 (49, resto 6), 9 (81, resto 7), 11 (121 > 97 → para). 121 > 97 → **primo**. Sólo 4 divisiones en lugar de 95.

Búsqueda del primer divisor impar d con d·d ≤ a. Si no se encuentra (d·d > a), es primo. Se compara `d * d ⩽ a` para no usar reales (`sqrt`). El 1 no es primo y el 2 es el único par primo.

C++ · PRIMO.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int a, d;
    bool primo;

    cout << "Entero a > 0: ";
    cin >> a;
    if (a < 2) {
        primo = false;                // 1 no es primo
    } else if (a % 2 == 0) {
        primo = (a == 2);             // el único par primo es el 2
    } else {
        d = 3;                        // sólo divisores impares hasta la raíz
        while ((d * d ⩽ a) && (a % d ≠ 0)) {
            d = d + 2;
        }
        primo = d * d > a;            // no se encontró divisor por debajo de la raíz
    }
    if (primo) { cout << a << " es primo" << endl; }
    else { cout << a << " no es primo" << endl; }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Entero a > 0: 97
97 es primo
```

Para 97 sólo se prueban 3, 5, 7 y 9 ($11^2 = 121 > 97$), frente a 95 divisiones del algoritmo inicial.

`Problema 3` · **Serie de e^x**

Suma `e^x = Σ x^i / i!` de dos formas: (1) con funciones Factorial y Potencia; (2) calculando cada término a partir del anterior. Termina tras n términos y compara con `exp`.

EXPLICACIÓN PASO A PASO

La serie

$$e^x = 1 + x + x^2/2! + x^3/3! + \ldots = \Sigma\, x^i / i!$$

Versión 1: cada término desde cero

Término i = `Potencia (x, i) / Factorial (i)`. Funciona, pero en cada vuelta recalcula la potencia (i multiplicaciones) y el factorial (i − 1 multiplicaciones) aunque la vuelta anterior ya casi los tenía.

Versión 2: cada término a partir del anterior

Divide un término entre el anterior:

$$t_{i+1} / t_i = (x^{i+1}/(i+1)!) / (x^i/i!) = x / (i + 1)$$

Así que $t_{i+1} = t_i \cdot x / (i + 1)$: una multiplicación y una división por vuelta.

1. Inicialización: suma = 0, término = 1 ($x^0/0! = 1$).
2. Tratamiento: suma = suma + término.
3. Avance: término = término · x / (i + 1).
4. Parada: tras n términos (se sabe cuántos → `for`).

i	término (x = 1)	suma tras sumarlo
0	1	1
1	1	2
2	0.5	2.5
3	0.1667	2.6667
4	0.0417	2.7083

Con 15 términos se llega a 2.71828182846, igual que `exp(1)`. Se usa `double` porque los factoriales crecen muy deprisa (13! ya no cabe en un `int`).

Término siguiente a partir del actual: $x^{i+1} / (i+1)! = (x^i / i!) \cdot x / (i + 1)$: una multiplicación y una división por vuelta, frente a las i + i que hace la versión 1. Elementos del esquema: inicialización (suma ← 0, término ← 1), tratamiento (suma ← suma + término), avance (término ← término · x / (i + 1)) y parada tras n términos.

C++ · EXP.CPP

```cpp
#include <iostream>
#include <cmath>
using namespace std;

/* Pre: n ≥ 0.  Post: n! */
double Factorial (int n)
{
    double f;
    int i;
    f = 1;
    for (i = 2; i ⩽ n; i = i + 1) {
        f = f * i;
    }
    return f;
}

/* Pre: y ≥ 0.  Post: x elevado a y */
double Potencia (double x, int y)
{
    double p;
    int i;
    p = 1;
    for (i = 1; i ⩽ y; i = i + 1) {
        p = p * x;
    }
    return p;
}

int main ( ) {
    double x, suma1, suma2, termino;
    int n, i;

    cout << "x y número de términos n: ";
    cin >> x >> n;

    // Versión 1: cada término desde cero con las dos funciones
    suma1 = 0;
    for (i = 0; i < n; i = i + 1) {
        suma1 = suma1 + Potencia (x, i) / Factorial (i);
    }

    // Versión 2: cada término a partir del anterior
    termino = 1;                // x^0 / 0!
    suma2 = 0;
    for (i = 0; i < n; i = i + 1) {
        suma2 = suma2 + termino;
        termino = termino * x / (i + 1);   // x^(i+1)/(i+1)! = x^i/i! · x/(i+1)
    }

    cout.precision(12);
    cout << "Versión 1: " << suma1 << endl;
    cout << "Versión 2: " << suma2 << endl;
    cout << "exp(x):    " << exp (x) << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
x y número de términos n: 1 15
Versión 1: 2.71828182846
Versión 2: 2.71828182846
exp(x):    2.71828182846
```

Con 15 términos las dos versiones coinciden con `exp (1)` en 12 cifras. Se usa `double` porque los factoriales crecen muy deprisa (13! ya no cabe en un `int`).

Problema 4 · **Sucesión $a_n = 3a_{n-1} - 2a_{n-3}$**

$a_0 = 1$, $a_1 = 2$, $a_2 = 3$. (a) El menor n tal que $a_n > m$. (b) El término n-ésimo.

EXPLICACIÓN PASO A PASO

Qué hace la recurrencia

$a_n = 3 \cdot a_{n-1} - 2 \cdot a_{n-3}$, con $a_0 = 1$, $a_1 = 2$, $a_2 = 3$

Para calcular un término hacen falta el **anterior** (n−1) y el de **tres antes** (n−3). Por eso se guardan los tres últimos en tres variables: a1 = a_{n-1}, a2 = a_{n-2}, a3 = a_{n-3}.

La «ventana» que se desplaza

1. Calcular el nuevo: an = 3·a1 − 2·a3.
2. Desplazar, del más viejo al más nuevo: a3 ← a2; a2 ← a1; a1 ← an. (En este orden, para no perder valores; es lo mismo que el intercambio de la práctica 2.)

n	Cuenta	a_n
3	3·3 − 2·1	7
4	3·7 − 2·2	17
5	3·17 − 2·3	45
6	3·45 − 2·7	121
7	3·121 − 2·17	329

- **(a) Menor n con $a_n > m$**: no se sabe cuántos términos harán falta → búsqueda con `while (an ⩽ m)`. Con m = 100: 45 ≤ 100, 121 > 100 → **n = 6**. Los tres primeros términos se tratan aparte porque no salen de la fórmula.
- **(b) Término n-ésimo**: se sabe cuántos pasos dar → `for (n = 3; n ⩽ k; …)`. a_6 = **121**.

Se usa `long long` porque los términos crecen casi como 3^n.

Hacen falta los **tres** términos anteriores, en tres variables que se «desplazan» en cada vuelta (como ult y penult en Fibonacci). Términos: 1, 2, 3, 7, 17, 45, 121, 329… (a) es una búsqueda (while), (b) un recorrido (for de 3 a n).

C++ · RECURRENCIA.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    long long m, a3, a2, a1, an;   // a3 = a(n-3), a2 = a(n-2), a1 = a(n-1)
    int n, k;

    // a) menor n con a(n) > m
    cout << "m (≥ 0): ";
    cin >> m;
    a3 = 1; a2 = 2; a1 = 3;       // a0, a1, a2
    if (1 > m) { n = 0; }
    else if (2 > m) { n = 1; }
    else if (3 > m) { n = 2; }
    else {
        n = 2;
        an = a1;
        while (an ≤ m) {
            n = n + 1;
            an = 3 * a1 - 2 * a3;
            a3 = a2; a2 = a1; a1 = an;   // desplazar la «ventana» de términos
        }
    }
    cout << "a) n = " << n << endl;

    // b) término n-ésimo
    cout << "n (≥ 0): ";
    cin >> k;
    a3 = 1; a2 = 2; a1 = 3;
    if (k == 0) { an = 1; }
    else if (k == 1) { an = 2; }
    else if (k == 2) { an = 3; }
    else {
        for (n = 3; n ≤ k; n = n + 1) {
            an = 3 * a1 - 2 * a3;
            a3 = a2; a2 = a1; a1 = an;
        }
    }
    cout << "b) a(" << k << ") = " << an << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
m (≥ 0): 100
a) n = 6
n (≥ 0): 6
b) a(6) = 121
```

Problema 5 · **Factorial**

Implementa el factorial estudiado en teoría.

EXPLICACIÓN PASO A PASO

Del RECORRIENDO al `for`

n! = 1 · 2 · 3 · … · n. En la notación: `fact ← 1; i RECORRIENDO [2, n] HACER fact ← fact · i`. En C++, un RECORRIENDO [a, b] es:

```
fact = 1;
for (i = 2; i ⩽ n; i = i + 1) {
    fact = fact * i;
}
```

1. **Inicialización** `fact = 1`: el elemento neutro del producto (como la suma empieza en 0).
2. El `for` tiene tres partes: valor inicial de i (2), condición para seguir (i ≤ n) y avance (i + 1).
3. Se empieza en 2 porque multiplicar por 1 no cambia nada.

Traza con n = 5: fact = 1 → ·2 = 2 → ·3 = 6 → ·4 = 24 → ·5 = **120**.

Con n = 0 o n = 1 el intervalo [2, n] está vacío: el bucle no entra y fact queda en 1, que es correcto (0! = 1! = 1) sin ningún caso aparte.

DESBORDAMIENTO

13! = 6 227 020 800 ya no cabe en un `int` (máximo 2 147 483 647) y el resultado sale negativo o absurdo sin avisar. Con `long long` cabe hasta 20!.

La versión con RECORRIENDO [2, n] pasa a un `for`. Con `long long` cabe hasta 20!; con `int`, sólo hasta 12!.

C++ · FACTORIAL.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int n, i;
    long long fact;

    cout << "n (⩾ 0): ";
    cin >> n;
    fact = 1;                               // 0! = 1
    for (i = 2; i ⩽ n; i = i + 1) {         // i RECORRIENDO [2, n]
        fact = fact * i;
    }
    cout << n << "! = " << fact << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
n (⩾ 0): 10
10! = 3628800
```

Problema 6 · **Fibonacci**

Implementa el término n-ésimo de Fibonacci estudiado en teoría ($f_0 = f_1 = 1$).

EXPLICACIÓN PASO A PASO

La sucesión

$$f_0 = f_1 = 1 \cdot f_i = f_{i-1} + f_{i-2}$$

Cada término es la suma de los **dos anteriores**, así que no hace falta guardar toda la sucesión: basta con tres variables, `penult`, `ult` y `fibo`.

Qué guarda cada variable

Antes de cada vuelta, `fibo` es el último término calculado (f_{i-1}) y `ult` el anterior (f_{i-2}). Se empieza con penult = 1, ult = 1, fibo = 2: f_0, f_1 y f_2.

1. `penult = ult;` → el que era último pasa a penúltimo (f_{i-2}).
2. `ult = fibo;` → el último calculado pasa a ser el último.
3. `fibo = ult + penult;` → se calcula el término nuevo, f_i.

Primero se «desplazan» los términos y después se calcula el nuevo. Si se cambia el orden (por ejemplo, `ult = fibo` antes que `penult = ult`), penult recibe el valor nuevo y se pierde uno que todavía hacía falta.

i	penult	ult	fibo = f_i
(inicio)	1	1	2
3	1	2	3
4	2	3	5
5	3	5	8
6	5	8	13

Sucesión: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 → $f_9 = 55$. Para n < 2 no se entra en el bucle y se da 1 directamente.

C++ · FIBONACCI.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    int n, i;
    long long penult, ult, fibo;

    cout << "n (≥ 0): ";
    cin >> n;
    if (n < 2) {
        fibo = 1;                          // f0 = f1 = 1
    } else {
        penult = 1; ult = 1; fibo = 2;     // f2
        for (i = 3; i ≤ n; i = i + 1) {
            penult = ult;
            ult = fibo;
            fibo = ult + penult;
        }
    }
    cout << "f(" << n << ") = " << fibo << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
n (≥ 0): 9
f(9) = 55
```

$1, 1, 2, 3, 5, 8, 13, 21, 34, \mathbf{55}$: $f_9 = 55$.

PRÁCTICA 6

Primer modelo de acceso secuencial

Secuencias marcadas leídas de la entrada estándar: esquemas de recorrido y búsqueda en C++.

6.1 Secuencias desde la entrada estándar

C++ no tiene el tipo «máquina secuencial»; usamos la entrada estándar como secuencia del primer modelo, terminada en una **marca** (MF) que no puede ser un dato.

RECORRIDO (ESQUEMA 1)

```
Comenzar (S);
{ inicialización }
MIENTRAS EA (S) ≠ MarcaFin HACER
  { tratamiento de EA }
  Avanzar (S)
FIN_MIENTRAS;
{ terminación }
```

C++

```
cin >> EA;                  // Comenzar (cin.get (EA) si
importan los blancos)
// inicialización
while (EA =≠ MF) {          // condición de continuación
    // tratamiento de EA
    cin >> EA;              // Avanzar
}
// terminación
```

Los ejemplos del boletín: pasar a mayúsculas un texto terminado en '#' (`char (int (EA) - Distancia)`, con `Distancia = int ('a') - int ('A')`), escribir el cuadrado de cada entero de [−99, 99] (marca −100) y contar las notas aprobadas (marca −1).

ERRATAS DEL BOLETÍN

En los ejemplos aparece el guion largo «–» en lugar del signo menos y `{cont = cont + 1};` con el punto y coma fuera de la llave; escríbelo `{ cont = cont + 1; }`. Y el contador del ejemplo 3 debería ser `int`, no `float`.

Para ver los esquemas paso a paso tienes el simulador de la máquina secuencial de la guía de teoría.

EXPLICADO DESPACIO

Los tres esquemas, con un ejemplo cada uno

- **Esquema 1** (H1 y H2: la vacía y el primero se tratan como los demás). Ejemplo: sumar todos. Inicializar con la solución de la vacía (suma = 0) y recorrer.
- **Esquema 2** (¬H1, H2: la vacía necesita un tratamiento aparte, pero el primero no). Ejemplo: la media (la de la vacía no existe).
- **Esquema 3** (¬H1, ¬H2: además el primer elemento es especial). Ejemplo: el máximo (se inicializa con el primero) o las sumas de adyacentes (el primero no tiene anterior).

Recorrido frente a búsqueda

Pregúntate: **¿puedo dar la respuesta sin mirar todos los elementos?** Si sí (el primer par, si aparece c x veces...), es una **búsqueda** y la condición del bucle tiene dos partes: `(EA =≠ MF) && NO encontrado`. Si hay que mirarlos todos (sumar, contar, la media), es un **recorrido** y la condición es sólo `EA =≠ MF`.

Tras una búsqueda siempre hay que mirar **por qué** se ha salido del bucle (¿se encontró o se acabó la secuencia?), y eso se sabe mirando si EA es la marca.

La plantilla en C++ (primer modelo, marca de fin)

```
inicialización;          // solución para la secuencia vacía
cin >> EA;               // Comenzar: el primero ya está disponible
while (EA =≠ MF) {       // en una búsqueda: && !Prop(EA)
    tratamiento de EA;   // mantener el invariante
    cin >> EA;           // Avanzar (siempre lo último del cuerpo)
}
tratamiento final;       // p. ej. la media si n > 0
```

6.2 Ejercicios de clase

Problema 1 · ¿Cuál de las dos versiones es correcta?

Dos versiones de un programa que trata enteros positivos con marca −1: la de la izquierda lee antes del bucle y al final del cuerpo; la de la derecha lee al principio del cuerpo. ¿Qué esquema se intenta usar? ¿Cuál es correcta? Predice qué pasará al compilar y ejecutar.

EXPLICACIÓN PASO A PASO

El esquema que se intenta: recorrido del primer modelo

En el primer modelo de acceso secuencial el primer elemento **está disponible nada más comenzar**, y el final se detecta cuando el elemento actual es la marca. Por eso el recorrido es:

1. **Comenzar**: leer el primer elemento (`cin >> EA;`).
2. **Mientras** EA no sea la marca:
3. tratar EA (escribirlo, contarlo);
4. **Avanzar**: leer el siguiente al **final** del cuerpo.

Por qué la versión de la derecha falla

- La primera vez compara `EA` con la marca **sin haberle dado valor**: el contenido es basura (comportamiento indefinido) y puede no entrar nunca.
- Cuando por fin lee el −1, ya está dentro del cuerpo: lo **escribe y lo cuenta** antes de volver a comprobar la condición. Trata la marca como un dato más y el contador sale uno de más.

Traza de la correcta con 4 8 15 −1: lee 4 → escribe, cont 1, lee 8 → escribe, cont 2, lee 15 → escribe, cont 3, lee −1 → sale. 3 datos.

> Ninguna de las dos compila tal cual porque usan `EA_S`, que no está declarada. El compilador lo dice con «was not declared in this scope».

- **Esquema**: recorrido del primer modelo (esquema 1): escribir cada dato y contarlos.
- **Al compilar: ninguna compila**, porque usan `EA_S`, que no está declarada (debería ser `EA`).
- **Corregido eso**, la de la izquierda es la correcta: Comenzar antes del bucle y Avanzar al final del cuerpo.
- La de la derecha **compara EA sin haberle dado valor** (comportamiento indefinido: puede que ni entre) y, cuando lee el −1, lo escribe y lo cuenta antes de comprobar la condición: **trata la marca como un dato** y cuenta uno de más.

C++ · VERSIÓN CORRECTA

```cpp
#include <iostream>
using namespace std;

int main( )
{
    const int MF = -1;
    int EA;
    int cont;
    cout << "Introduce una serie de enteros positivos, para terminar -1" << endl;
    cin >> EA;                    // Comenzar
    cont = 0;
    while (EA =≠ MF) {
        cout << EA << ", ";       // tratamiento (EA_S no existe: es EA)
        cont = cont + 1;
        cin >> EA;                // Avanzar
    }
    cout << endl << "El número de datos recogidos es: " << cont << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Introduce una serie de enteros positivos, para terminar -1
4 8 15 -1
4, 8, 15,
El número de datos recogidos es: 3
```

Problema 2 · **Sumas parciales**

Dada una secuencia de enteros positivos con marca −1, escribe S1, S1+S2, S1+S2+S3... Para 1, 4, 5, 3, 5, 6 → 1, 5, 10, 13, 18, 24.

EXPLICACIÓN PASO A PASO

Qué se pide

Cada número que se escribe es la suma de **todos los leídos hasta ahora**: con 1, 4, 5 → 1, 1 + 4 = 5, 5 + 5 = 10.

El invariante: suma = Σ Piz

«Piz» es la parte izquierda: los elementos ya tratados. Si mantenemos que `suma` es siempre la suma de Piz, al tratar el elemento actual basta hacer `suma ← suma + EA` y escribirla.

1. **Inicialización**: Piz está vacía al principio, y la suma de la secuencia vacía es 0 → suma = 0.
2. **Tratamiento**: suma ← suma + EA; escribir suma.
3. **Avance**: leer el siguiente.

EA	suma	Se escribe
1	1	1
4	5	5
5	10	10
3	13	13
5	18	18
6	24	24
−1	—	fin

Es el **esquema 1** (H1 y H2): la secuencia vacía no necesita nada especial (no escribe nada) y el primer elemento se trata como los demás.

Recorrido con H1 y H2 (esquema 1). Inv: suma = Σ Piz. Inicialización: suma de la vacía = 0. Tratamiento: suma ← suma + EA y registrar suma.

C++ · PARCIALES.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const int MF = -1;
    int EA;        // elemento actual
    int suma;      // suma de la parte izquierda

    cout << "Enteros positivos, para terminar -1" << endl;
    suma = 0;                    // suma de la secuencia vacía
    cin >> EA;                   // Comenzar
    while (EA =≠ MF) {
        suma = suma + EA;        // suma de Piz + EA
        cout << suma << " ";     // Registrar
        cin >> EA;               // Avanzar
    }
    cout << endl;
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Enteros positivos, para terminar -1
1 4 5 3 5 6 -1
1 5 10 13 18 24
```

Problema 3 · **Sumas de elementos adyacentes**

De S = [S1, ..., Sn] obtén R = [S1+S2, S2+S3, ..., Sn−1+Sn]. Si S está vacía o tiene un único elemento, «La secuencia está vacía». ¿Qué tipo de problema es? ¿Qué esquema? Postcondición, invariante y diseño.

EXPLICACIÓN PASO A PASO

Por qué es el esquema 3

Cada suma usa **dos** elementos seguidos, el anterior y el actual. El primer elemento no tiene anterior, así que no produce ninguna suma: **no se puede tratar como los demás** (¬H2). Y la secuencia vacía necesita un mensaje propio (¬H1). Además, con un solo elemento tampoco hay pares.

Procedimiento

1. Leer el primero. Si es la marca → «vacía».
2. Si no, guardarlo en `ant` (tratamiento del primer elemento: sólo recordarlo) y leer el siguiente.
3. Si ese es la marca → sólo había uno → «vacía» (según el enunciado).
4. Si no, repetir: escribir `ant + EA` ; `ant ← EA` ; leer el siguiente; hasta llegar a la marca.

El invariante: **lo escrito = las sumas adyacentes de Piz** y **ant = el último de Piz**. Por eso, al tratar EA, la única suma nueva es ant + EA.

Traza con 3 1 4 1 5: ant = 3; EA = 1 → 4; ant = 1, EA = 4 → 5; ant = 4, EA = 1 → 5; ant = 1, EA = 5 → 6. Salida **4 5 5 6**: con n elementos salen n − 1 sumas.

- **Tipo**: recorrido (todos los pares adyacentes).
- **Esquema 3** (¬H1 y ¬H2): la vacía es un caso aparte y el primer elemento no genera salida (no tiene anterior). Aquí además hay que distinguir la secuencia de un solo elemento.
- **Post**: R = SumasAdy (S). **Inv**: lo escrito = SumasAdy (Piz) y ant = último (Piz).
- **Definición inductiva**: SumasAdy ([e]) = [ ]; SumasAdy (T·u·e) = SumasAdy (T·u) & [u + e].

C++ · ADYACENTES.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const int MF = -1;
    int EA, ant;     // elemento actual y el anterior (último de Piz)

    cout << "Enteros positivos, para terminar -1" << endl;
    cin >> EA;                                  // Comenzar
    if (EA == MF) {
        cout << "La secuencia está vacía" << endl;
    } else {
        ant = EA;                               // tratamiento del primer elemento
        cin >> EA;                              // Avanzar
        if (EA == MF) {
            cout << "La secuencia está vacía" << endl;   // sólo un elemento
        } else {
            do {
                cout << ant + EA << " ";        // Registrar la suma del par (ant, EA)
                ant = EA;
                cin >> EA;                      // Avanzar
            } while (EA != MF);
            cout << endl;
        }
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Enteros positivos, para terminar -1
3 1 4 1 5 -1
4 5 5 6
```

Se ha supuesto, como en el resto de la práctica, enteros positivos con marca −1 (el enunciado no fija la marca).

Problema 4 · **Primer número par**

Secuencia de enteros de [−99, 99] con marca −1: ¿cuál es el primer par, si hay alguno? ¿Recorrido o búsqueda? ¿Condición para detener?

EXPLICACIÓN PASO A PASO

Recorrido o búsqueda

No hace falta mirar todos los números: en cuanto aparece el primer par, ya tenemos la respuesta. Eso es una **búsqueda**, que se para por una de dos razones: se encuentra lo buscado o se llega al final.

`continuar mientras (EA ≠ MF) Y (EA no es par)`

1. Leer el primero.
2. Mientras no sea la marca y sea impar: leer el siguiente. (El cuerpo sólo avanza: no hay nada que tratar.)
3. Al salir, mirar **por qué** se ha salido: si EA es la marca, no había pares; si no, EA es el primer par.

IMPAR CON NEGATIVOS

En C++, `-3 % 2` vale **−1**, no 1 (el resto lleva el signo del dividendo). Por eso «es impar» se escribe `EA % 2 ≠ 0`, que funciona con positivos y negativos; `EA % 2 == 1` fallaría con −3.

Traza con 7 −3 15 −8 4: 7 impar → sigue; −3 impar → sigue; 15 → sigue; −8 % 2 = 0 → par → para. Primer par: **−8**. El 4 no llega a leerse.

Búsqueda: se para en cuanto aparece un par (o en la marca). Condición de continuación: `(EA ≠ MF) && (EA % 2 ≠ 0)`. Al salir, se mira si se paró en la marca (no hay) o en un par.

DOS DETALLES

- Con negativos, `-3 % 2` vale −1, así que «impar» es `EA % 2 ≠ 0`, no `EA % 2 == 1`.
- El −1 está dentro de [−99, 99] pero es la marca: no puede aparecer como dato (y al ser impar no afecta a la búsqueda).

C++ · PRIMERPAR.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const int MF = -1;
    int EA;

    cout << "Enteros en [-99, 99], para terminar -1" << endl;
    cin >> EA;                                  // Comenzar
    while ((EA ≠ MF) && (EA % 2 ≠ 0)) {       // continuar: no es el final Y no es par
        cin >> EA;                              // Avanzar
    }
    if (EA == MF) {
        cout << "No hay ningún número par" << endl;
    } else {
        cout << "El primer par es " << EA << endl;
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Enteros en [-99, 99], para terminar -1
7 -3 15 -8 4 -1
El primer par es -8
```

6.3 Trabajo personal

Problema 1 · **Marcador de tenis**

Una función que, dado el marcador y quién gana el punto, muestre el nuevo marcador; y un algoritmo que simule el partido con una secuencia de '1' y '2' terminada en '0'. Gana el partido quien llegue a seis juegos. Para «112211221»: quince–nada, treinta–nada, treinta–quince, iguales a treinta, cuarenta–treinta, juego para el jugador 1, nada–quince, nada–treinta, quince–treinta.

EXPLICACIÓN PASO A PASO

Cómo se cuenta en el tenis (lo que hay que programar)

- Dentro de un juego se cuentan puntos: 0, 1, 2, 3 se dicen «nada, quince, treinta, cuarenta».
- Se gana el juego con **4 puntos o más y 2 de ventaja**.
- Si los dos llegan a 3 (cuarenta), a partir de ahí: empate = «iguales a cuarenta»; uno de diferencia = «ventaja».
- Gana el partido quien llega a 6 juegos.

El diseño

1. Un `struct Marcador` con los puntos y los juegos de cada jugador.
2. `AnotarPunto (Marcador &m, char ganador)` : modifica el marcador → acción con m por referencia (dato–resultado). Suma el punto y después decide qué escribir, en este orden: ¿juego para el 1? ¿juego para el 2? ¿iguales a cuarenta? ¿ventaja? ¿iguales (a quince, a treinta)? y, si no, «A–B».
3. Una función `Nombre (p)` traduce 0–3 a «nada»...«cuarenta» con un `switch` .
4. `main` es un recorrido de la secuencia de '1' y '2' que también para si alguien llega a 6 juegos: la condición tiene las dos cosas.

Traza de «1122112210»: 1–0 quince-nada; 2–0 treinta-nada; 2–1 treinta-quince; 2–2 iguales a treinta; 3–2 cuarenta-treinta; 4–2 (4 puntos y 2 de ventaja) → juego para el 1; luego 0–1, 0–2, 1–2.

El orden de las preguntas importa: «iguales a cuarenta» (3–3, 4–4...) tiene que ir antes que «iguales a treinta», que es el caso general de empate.

Como modifica el marcador, en la notación es una **acción** (dato–resultado marcador, dato ganador). Puntos 0, 1, 2, 3 = nada, quince, treinta, cuarenta; con 3–3 o más: «iguales a cuarenta» y «ventaja»; se gana el juego con 4 puntos o más y 2 de diferencia. El algoritmo principal es un recorrido que además **para** si alguien llega a seis juegos (una búsqueda).

C++ · TENIS.CPP

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Marcador {
    int puntos1, puntos2;     // puntos del juego en curso (0, 1, 2, 3, 4…)
    int juegos1, juegos2;
};

/* Post: nombre del tanteo correspondiente a p puntos (p ⩽ 3) */
string Nombre (int p)
{
    string s;
    switch (p) {
        case 0: s = "nada"; break;
        case 1: s = "quince"; break;
        case 2: s = "treinta"; break;
        default: s = "cuarenta"; break;
    }
    return s;
}

/* Pre: ganador es '1' o '2'
   Post: m actualizado con el punto y el nuevo marcador escrito en pantalla */
void AnotarPunto (Marcador &m, char ganador)
{
    if (ganador == '1') { m.puntos1 = m.puntos1 + 1; }
    else { m.puntos2 = m.puntos2 + 1; }

    if ((m.puntos1 ⩾ 4) && (m.puntos1 - m.puntos2 ⩾ 2)) {
        m.juegos1 = m.juegos1 + 1;  m.puntos1 = 0;  m.puntos2 = 0;
        cout << "juego para el jugador 1 (juegos " << m.juegos1 << "-" << m.juegos2 << ")" << endl;
    } else if ((m.puntos2 ⩾ 4) && (m.puntos2 - m.puntos1 ⩾ 2)) {
        m.juegos2 = m.juegos2 + 1;  m.puntos1 = 0;  m.puntos2 = 0;
        cout << "juego para el jugador 2 (juegos " << m.juegos1 << "-" << m.juegos2 << ")" << endl;
    } else if ((m.puntos1 ⩾ 3) && (m.puntos1 == m.puntos2)) {
        cout << "iguales a cuarenta" << endl;
    } else if ((m.puntos1 ⩾ 3) && (m.puntos2 ⩾ 3)) {
        if (m.puntos1 > m.puntos2) { cout << "ventaja para el jugador 1" << endl; }
        else { cout << "ventaja para el jugador 2" << endl; }
    } else if (m.puntos1 == m.puntos2) {
        cout << "iguales a " << Nombre (m.puntos1) << endl;
    } else {
        cout << Nombre (m.puntos1) << "-" << Nombre (m.puntos2) << endl;
    }
}

int main ( ) {
    const char MF = '0';
    const int JUEGOS = 6;
    Marcador m;
    char EA;

    m.puntos1 = 0; m.puntos2 = 0; m.juegos1 = 0; m.juegos2 = 0;
    cin >> EA;                                                  // Comenzar
    while ((EA ≠ MF) && (m.juegos1 < JUEGOS) && (m.juegos2 < JUEGOS)) {
        AnotarPunto (m, EA);
        cin >> EA;                                              // Avanzar
    }
    if (m.juegos1 == JUEGOS) { cout << "Partido para el jugador 1" << endl; }
    else if (m.juegos2 == JUEGOS) { cout << "Partido para el jugador 2" << endl; }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
1122112210
quince-nada
treinta-nada
treinta-quince
iguales a treinta
cuarenta-treinta
juego para el jugador 1 (juegos 1-0)
nada-quince
nada-treinta
quince-treinta
```

Problema 2 · **Ventanillas de la caja de ahorros**

N ventanillas. Secuencia de enteros terminada en 0: «1» = llega un cliente («Pase» si hay ventanilla libre; si no, «Espere por favor» y a la cola); «2» = queda libre una ventanilla («Pase» si hay cola). Al principio todas libres.

EXPLICACIÓN PASO A PASO

Qué hay que recordar en cada momento

No hace falta saber qué cliente está en qué ventanilla, sólo **cuántas ventanillas están libres** y **cuántos clientes esperan**. Dos contadores bastan.

1. **Inicialización**: libres = N, cola = 0.
2. Llega un cliente («1»): si hay libres, ocupa una (libres − 1) y «Pase»; si no, a la cola (cola + 1) y «Espere por favor».
3. Se libera una ventanilla («2»): si hay cola, el primero de la cola la ocupa (cola − 1) y «Pase» (libres no cambia: se libera y se vuelve a ocupar); si no hay cola, libres + 1.

El invariante explica por qué funciona: **si hay gente en la cola es que no hay ninguna libre** (cola > 0 ⇒ libres = 0).

EA	libres	cola	Salida
inicio	3	0	
1	2	0	Pase
1	1	0	Pase
1	0	0	Pase
1	0	1	Espere
2	0	0	Pase
1	0	1	Espere
2	0	0	Pase
2	1	0	
2	2	0	

Recorrido con dos contadores: ventanillas libres y clientes en cola. Inv: libres + ocupadas = N y cola > 0 ⇒ libres = 0.

C++ · VENTANILLAS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const int N = 3;        // número de ventanillas
    const int MF = 0;
    int EA;
    int libres, cola;       // ventanillas libres y clientes esperando

    libres = N;
    cola = 0;
    cin >> EA;                                  // Comenzar
    while (EA ≠ MF) {
        if (EA == 1) {                          // llega un cliente
            if (libres > 0) {
                libres = libres - 1;
                cout << "Pase" << endl;
            } else {
                cola = cola + 1;
                cout << "Espere por favor" << endl;
            }
        } else {                                // EA == 2: se libera una ventanilla
            if (cola > 0) {
                cola = cola - 1;                // la ocupa el primero de la cola
                cout << "Pase" << endl;
            } else {
                libres = libres + 1;
            }
        }
        cin >> EA;                              // Avanzar
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
1 1 1 1 2 1 2 2 2 0
Pase
Pase
Pase
Espere por favor
Pase
Espere por favor
Pase
```

Problema 3 · **Estadística de pesos**

Pesos con marca −1.0: cuántos pesan menos de 40 kg, entre 40 y 50 y más de 50, y la media.

EXPLICACIÓN PASO A PASO

Un recorrido con cuatro acumuladores

Hay que mirar todos los pesos, así que es un **recorrido**. Para cada uno se hacen dos cosas: decidir en qué grupo cae (un análisis de casos) y sumarlo para la media.

1. **Inicialización:** los tres contadores y la suma a 0 (la respuesta correcta para «aún no he visto ningún peso»).
2. **Tratamiento:** si < 40 → menos40 + 1; si no, si ≤ 50 → entre40y50 + 1; si no → mas50 + 1. Después, suma + EA.
3. **Tras el bucle:** n = menos40 + entre40y50 + mas50 y la media = suma / n.

La media de la secuencia vacía

Si no hay ningún peso, n = 0 y suma / n sería dividir entre 0. Dentro del bucle no hace falta nada especial (los contadores a 0 ya son correctos), pero al final sí: **la media sólo se calcula si n > 0**.

Comprobación con 38.5, 45, 52.3, 61, 40, 50: < 40 → 38.5 (1); entre 40 y 50 → 45, 40, 50 (3); > 50 → 52.3, 61 (2). Suma 286.8 / 6 = **47.8**.

> Los límites: 40 y 50 cuentan como «entre 40 y 50» por cómo están escritas las condiciones (`< 40` y `≤ 50`). Si el enunciado los quisiera en otro grupo, se cambia el operador.

- **Recorrido** (hay que mirar todos).
- La vacía no necesita tratamiento aparte **dentro** del bucle (H1 y H2: contadores a 0), pero sí al final: la media de la vacía no existe.
- **Tras la iteración:** calcular la media sólo si n > 0.

C++ · PESOS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const float MF = -1.0;
    float EA, suma;
    int menos40, entre40y50, mas50, n;

    menos40 = 0; entre40y50 = 0; mas50 = 0; suma = 0;
    cout << "Pesos, para terminar -1" << endl;
    cin >> EA;                              // Comenzar
    while (EA =/= MF) {
        if (EA < 40) { menos40 = menos40 + 1; }
        else if (EA ≤ 50) { entre40y50 = entre40y50 + 1; }
        else { mas50 = mas50 + 1; }
        suma = suma + EA;
        cin >> EA;                          // Avanzar
    }
    n = menos40 + entre40y50 + mas50;
    cout << "Menos de 40 kg: " << menos40 << endl;
    cout << "Entre 40 y 50 kg: " << entre40y50 << endl;
    cout << "Más de 50 kg: " << mas50 << endl;
    if (n > 0) { cout << "Peso medio: " << suma / n << endl; }
    else { cout << "No hay alumnos" << endl; }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Pesos, para terminar -1
38.5 45 52.3 61 40 50 -1.0
Menos de 40 kg: 1
Entre 40 y 50 kg: 3
Más de 50 kg: 2
Peso medio: 47.8
```

Problema 4 · ¿Al menos x apariciones de c?

Texto terminado en '#' (sin contar blancos ni saltos de línea). Se leen antes x y c. Salida «sí» o «no».

EXPLICACIÓN PASO A PASO

Por qué es una búsqueda

Preguntan si hay **al menos** x apariciones. En cuanto se encuentra la x-ésima, la respuesta es «sí» y no hace falta leer el resto: se busca **la x-ésima aparición de c**.

El invariante y la condición

noc = número de veces que c aparece en Piz (lo ya leído). El bucle tiene que parar si se llega a la marca o si el carácter actual es c y ya llevamos x − 1 (este sería el x-ésimo):

```
continuar mientras (EA ≠ '#') Y NO (EA = c Y noc = x - 1)
```

Por De Morgan, NO (A Y B) = (NO A) O (NO B), que es lo que está en el código: `(EA ≠ c) || (noc ≠ x - 1)`.

1. noc = 0; leer el primero.
2. Mientras no haya que parar: si EA = c, noc + 1 (para mantener el invariante); leer el siguiente.
3. Al salir: si EA no es la marca, se paró por haber encontrado la x-ésima → «sí»; si es la marca → «no».

Traza con x = 3, c = 'a' y «la casa de mi abuela#»: l, a (noc 1), c, a (2), s, a → es 'a' y noc = 2 = x − 1 → para: **sí**.

Se usa `cin >>` porque el enunciado dice que no cuentan blancos ni saltos de línea, y `>>` se los salta solo.

- **Búsqueda** de la x-ésima aparición de c: no hace falta leer el resto en cuanto se encuentra.
- **Fin**: EA = '#' o (EA = c y noc = x − 1).
- **Tratamiento** para mantener el invariante (noc = apariciones de c en Piz): si EA = c, noc ← noc + 1.
- Es el ejemplo del tema 4 con `cin >>`, que ya se salta blancos y saltos de línea.

C++ · OCURRENCIAS.CPP

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const char MF = '#';
    int x, noc;      // ocurrencias pedidas y ocurrencias en Piz
    char c, EA;

    cout << "Número de veces y carácter: ";
    cin >> x >> c;
    cout << "Texto terminado en #: ";
    noc = 0;
    cin >> EA;                                          // Comenzar (>> se salta los blancos)
    while ((EA ≠ MF) && ((EA ≠ c) || (noc ≠ x - 1))) {
        if (EA = c) { noc = noc + 1; }                  // mantener Inv: noc = NroOcur (c, Piz)
        cin >> EA;                                      // Avanzar
    }
    if (EA ≠ MF) { cout << "sí" << endl; }
    else { cout << "no" << endl; }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Número de veces y carácter: 3 a
Texto terminado en #: la casa de mi abuela #
sí
```

Problema 5 · **Igual a la suma de los anteriores**

Enteros de [−99, 99] con marca −100: el primero que sea igual a la suma de los anteriores. (a) Suponiendo que la suma de la vacía es 0; (b) suponiendo que no está definida.

EXPLICACIÓN PASO A PASO

Una búsqueda cuya propiedad depende de lo anterior

Se busca el primer EA que cumple **EA = suma de los anteriores**. Para comprobarlo sin volver atrás, se lleva la suma de Piz en una variable (invariante: suma = Σ Piz).

```
continuar mientras (EA ≠ MF) Y (EA ≠ suma)
```

(a) La suma de la secuencia vacía es 0

1. suma = 0; leer el primero. El primero ya es candidato: si vale 0, es igual a la suma de «ninguno».
2. Mientras no haya que parar: suma ← suma + EA; leer el siguiente.
3. Al salir: si es la marca, no hay ninguno; si no, EA es el buscado.

Traza con 3 5 2 10 20: suma 0, EA 3 ≠ 0 → suma 3; 5 ≠ 3 → 8; 2 ≠ 8 → 10; 10 = 10 → para: **10** (3 + 5 + 2 = 10).

(b) La suma de la vacía no está definida

Entonces el primer elemento **no puede ser candidato** (no tiene anteriores con los que compararse): se trata aparte (¬H1, ¬H2). Se lee, se usa como suma inicial y la búsqueda empieza en el segundo.

Traza con 0 4 1 5 10: suma = 0 (el primero); 4 ≠ 0 → 4; 1 ≠ 4 → 5; 5 = 5 → **5**. Con la versión (a), en cambio, el 0 inicial ya sería la respuesta.

- **Búsqueda** con una propiedad que depende de Piz.
- **Post:** (EA = MF o EA = Σ Piz) y ningún elemento anterior cumple la propiedad. **Inv:** suma = Σ Piz y ninguno de Piz es igual a la suma de los suyos anteriores.
- **Continuación:** (EA ≠ MF) y (EA ≠ suma). **Definición inductiva:** Σ [ ] = 0; Σ (T·e) = Σ T + e.

C++ · (A) LA SUMA DE LA VACÍA ES 0

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const int MF = -100;
    int EA, suma;    // suma de la parte izquierda

    cout << "Enteros en [-99, 99], para terminar -100" << endl;
    suma = 0;                                   // suma de la secuencia vacía
    cin >> EA;                                  // Comenzar
    while ((EA =/= MF) && (EA =/= suma)) {
        suma = suma + EA;                       // Inv: suma = Σ Piz
        cin >> EA;                              // Avanzar
    }
    if (EA == MF) { cout << "Ningún elemento es igual a la suma de los anteriores" << endl; }
    else { cout << "El primero es " << EA << endl; }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Enteros en [-99, 99], para terminar -100
3 5 2 10 20 -100
El primero es 10
```

(b) Si la suma de la vacía no está definida, el primer elemento no es candidato: se trata aparte (¬H1, ¬H2) y la búsqueda empieza en el segundo.

C++ · (B) EL PRIMERO NO ES CANDIDATO

```cpp
#include <iostream>
using namespace std;

int main ( ) {
    const int MF = -100;
    int EA, suma;

    cout << "Enteros en [-99, 99], para terminar -100" << endl;
    cin >> EA;                                  // Comenzar
    if (EA == MF) {
        cout << "Secuencia vacía" << endl;
    } else {
        suma = EA;                              // el primero no es candidato
        cin >> EA;                              // Avanzar
        while ((EA != MF) && (EA != suma)) {
            suma = suma + EA;
            cin >> EA;
        }
        if (EA == MF) { cout << "Ningún elemento es igual a la suma de los anteriores" << endl; }
        else { cout << "El primero es " << EA << endl; }
    }
    return 0;
}
```

EJECUCIÓN DE EJEMPLO

```
Enteros en [-99, 99], para terminar -100
0 4 1 5 10 -100
El primero es 5
```

Basado en los boletines de las prácticas 1 a 6 de la asignatura. Cada programa se ha compilado con g++ -Wall -Wextra sin avisos y las ejecuciones de ejemplo son salidas reales.